
TRABAJO FIN DE MÁSTER

MÁSTER EN CIBERSEGURIDAD

LinuxRecon

Herramienta para la recopilación automatizada de información en sistemas Linux



ÍNDICE

1. ESTADO DEL ARTE.....	7
1.1 La auditoría de seguridad en sistemas Linux.....	7
1.2 Clasificación de las herramientas de auditoría de seguridad.....	8
1.3 Herramientas de reconocimiento (Information Gathering).....	9
1.4 Herramientas de auditoría del sistema.....	9
1.5 Escáneres de vulnerabilidades.....	10
1.6 Plataformas SIEM y monitorización continua.....	11
1.7 Comparativa de las soluciones existentes.....	11
1.8 Posicionamiento de LinuxRecon.....	12
2. MOTIVACIÓN.....	15
3. HIPÓTESIS.....	19
3.1 Planteamiento de la hipótesis.....	19
3.2 Propuesta de solución.....	20
3.3 Objetivos del proyecto.....	21
3.4 Alcance de la hipótesis.....	22
4. TESIS.....	25
4.1 Propuesta de la solución.....	25
4.2 Objetivos funcionales.....	27
4.3 Metodología de desarrollo.....	31
4.4 Planificación del proyecto.....	33
4.5 Tecnologías utilizadas.....	35
4.6 Diseño de la arquitectura.....	38
4.7 Flujo de funcionamiento.....	41
4.8 Alcance, limitaciones y plan de evolución.....	45

5. DEMOSTRACIÓN.....	52
5.1 Implementación.....	52
5.2 Validación y pruebas.....	54
5.3 Matriz global de validación.....	56
5.4 Evaluación básica de eficiencia.....	57
5.5 Análisis global de los resultados.....	59
5.6 Relación entre hipótesis y resultados.....	59
5.7 Limitaciones de la demostración.....	59
5.8 Conclusiones de la demostración.....	59
6. CONCLUSIONES Y LÍNEAS FUTURAS.....	60
6.1 Conclusiones generales.....	60
6.2 Cumplimiento de los objetivos.....	60
6.3 Contraste de la hipótesis.....	60
6.4 Aportaciones del proyecto.....	60
6.5 Seguridad de la solución.....	61
6.6 Eficiencia y reproducibilidad.....	61
6.7 Limitaciones.....	61
6.8 Líneas futuras.....	61
6.9 Evolución propuesta.....	61
6.10 Conclusión final.....	62
7. BIBLIOGRAFÍA.....	63
7.1 Ingeniería del software y metodología.....	63
7.2 Sistemas Linux y administración.....	63
7.3 Tecnologías utilizadas.....	63
7.4 Herramientas de auditoría y ciberseguridad.....	63
7.5 Informes y contexto de amenazas.....	64
8. ANEXOS.....	65
8.1 ANEXO A. Código fuente final de LinuxRecon (my.py).....	65
8.2 ANEXO B. Estructura y evidencia del informe final.....	67
8.3 ANEXO C. Evidencias de validación.....	68

ÍNDICE DE FIGURAS

Figura 1. Clasificación funcional de las principales herramientas utilizadas durante una auditoría de seguridad en sistemas Linux. Fuente: elaboración propia.....	8
Figura 2. Posicionamiento de LinuxRecon dentro del ciclo de una auditoría de seguridad. Fuente: elaboración propia.....	14
Figura 3. Motivación del desarrollo de LinuxRecon y problema que resuelve. Fuente: elaboración propia.....	18
Figura 4. Hipótesis de investigación y propuesta de solución mediante LinuxRecon. Fuente: elaboración propia.....	24
Figura 5. Propuesta de valor de LinuxRecon como herramienta de apoyo a la recopilación inicial. Fuente: elaboración propia.....	27
Figura 6. Relación entre los objetivos funcionales y los componentes principales de LinuxRecon. Fuente: elaboración propia.....	29
Figura 7. Metodología incremental aplicada al desarrollo de LinuxRecon. Fuente: elaboración propia.....	32
Figura 8. Organización general de las fases del proyecto LinuxRecon. Fuente: elaboración propia.....	34
Figura 9. Cronograma general del proyecto LinuxRecon. Fuente: elaboración propia.....	35
Figura 10. Tecnologías y fuentes de información utilizadas por LinuxRecon. Fuente: elaboración propia.....	37
Figura 11. Arquitectura lógica de la versión final de LinuxRecon. Fuente: elaboración propia.....	39

Figura 12. Flujo principal de ejecución de LinuxRecon. Fuente: elaboración propia.....	43
Figura 13. Estructura lógica de los módulos de recopilación de LinuxRecon. Fuente: elaboración propia.....	45
Figura 14. Síntesis del estado final y evolución propuesta de LinuxRecon. Fuente: elaboración propia.....	50
Figura 15. Organización lógica de la implementación final de LinuxRecon. Fuente: elaboración propia.....	52
Figura 16. Flujo de recopilación, síntesis y protección de report.txt. Fuente: elaboración propia.....	54
Figura 17. Comparación entre datos del sistema y resultados registrados por LinuxRecon. Fuente: elaboración propia.....	55
Figura 18. Puertos y sockets recopilados durante la validación. Fuente: elaboración propia.	55
Figura 19. Información de red obtenida durante las pruebas. Fuente: elaboración propia.....	56
Figura 20. Evidencia de creación y consulta del informe report.txt. Fuente: elaboración propia.....	56
Figura 21. Fragmento representativo del informe final generado por LinuxRecon. Fuente: elaboración propia.....	58

ÍNDICE DE TABLAS

Tabla 1. Comparativa de herramientas de ciberseguridad y posicionamiento de LinuxRecon. Fuente: elaboración propia.....	12
Tabla 2. Requisitos funcionales de LinuxRecon.....	29
Tabla 3. Correspondencia entre objetivos y componentes.....	30
Tabla 4. Principales fuentes y utilidades utilizadas por LinuxRecon.....	36
Tabla 5. Componentes principales de LinuxRecon.....	40
Tabla 6. Trazabilidad entre requisitos y arquitectura.....	40
Tabla 7. Funcionalidades implementadas.....	45
Tabla 8. Seguridad de la versión final.....	47
Tabla 9. Métricas de validación.....	48
Tabla 10. Síntesis del estado final y evolución.....	50
Tabla 11. Funciones principales implementadas en LinuxRecon. Fuente: elaboración propia.	53
Tabla 12. Entorno utilizado durante la validación de LinuxRecon. Fuente: elaboración propia.....	54
Tabla 13. Matriz global de validación de LinuxRecon. Fuente: elaboración propia.....	57
Tabla 14. Medición temporal de referencia de la versión base. Fuente: elaboración propia.	57
Tabla 15. Características del informe final ampliado. Fuente: elaboración propia.....	57
Tabla 16. Evolución propuesta de LinuxRecon. Fuente: elaboración propia.....	61

1. ESTADO DEL ARTE

1.1 La auditoría de seguridad en sistemas Linux

La auditoría de seguridad constituye una de las actividades fundamentales dentro de la ciberseguridad, ya que permite evaluar el estado de un sistema informático con el objetivo de identificar configuraciones inseguras, servicios expuestos, debilidades de seguridad y posibles vectores de ataque antes de que puedan ser aprovechados por un actor malicioso. Su finalidad principal es conocer el nivel de exposición de una infraestructura y proporcionar la información necesaria para aplicar medidas correctivas que reduzcan el riesgo de compromiso.

En la actualidad, los sistemas operativos Linux desempeñan un papel esencial en el ámbito tecnológico. Una gran parte de los servidores corporativos, plataformas cloud, centros de datos, infraestructuras críticas y dispositivos del Internet de las Cosas (IoT) utilizan distribuciones Linux debido a su estabilidad, flexibilidad y elevado grado de personalización. Esta amplia adopción ha convertido a Linux en un objetivo habitual de ataques dirigidos tanto a organizaciones públicas como privadas.

Como consecuencia, la realización periódica de auditorías de seguridad se ha convertido en una práctica imprescindible dentro de cualquier estrategia de protección de infraestructuras. Estas auditorías permiten detectar errores de configuración, servicios innecesarios, permisos incorrectos, cuentas de usuario mal gestionadas o aplicaciones potencialmente vulnerables antes de que puedan ser explotadas.

Aunque existen diferentes metodologías de auditoría, la mayoría comparten un flujo de trabajo similar compuesto por varias fases consecutivas. En primer lugar se realiza la recopilación de información (*Information Gathering*), donde se obtiene una visión inicial del sistema analizado. Posteriormente se lleva a cabo la enumeración de recursos, el análisis de vulnerabilidades, la evaluación de riesgos y, finalmente, la aplicación de medidas de mitigación y endurecimiento (*hardening*).

Entre todas estas etapas, la fase de recopilación de información constituye el punto de partida sobre el que se apoyan las restantes. Cuanto más completa y precisa sea la información obtenida inicialmente, mayor será la capacidad del analista para identificar riesgos y planificar el resto de la auditoría.

1.2 Clasificación de las herramientas de auditoría de seguridad

El creciente número de herramientas desarrolladas para la auditoría de sistemas ha dado lugar a soluciones muy especializadas, cada una orientada a resolver una parte concreta del proceso de evaluación de seguridad. Aunque algunas plataformas integran funcionalidades pertenecientes a varias categorías, la mayoría pueden clasificarse atendiendo al objetivo principal para el que fueron diseñadas.

Desde un punto de vista funcional, las herramientas de auditoría pueden agruparse en cuatro grandes categorías:

- Herramientas de reconocimiento o *Information Gathering*.
- Herramientas de auditoría del sistema.
- Escáneres de vulnerabilidades.
- Plataformas de monitorización y gestión de eventos de seguridad (SIEM).

Cada una de estas categorías responde a necesidades distintas dentro del proceso de auditoría y presenta ventajas e inconvenientes específicos.

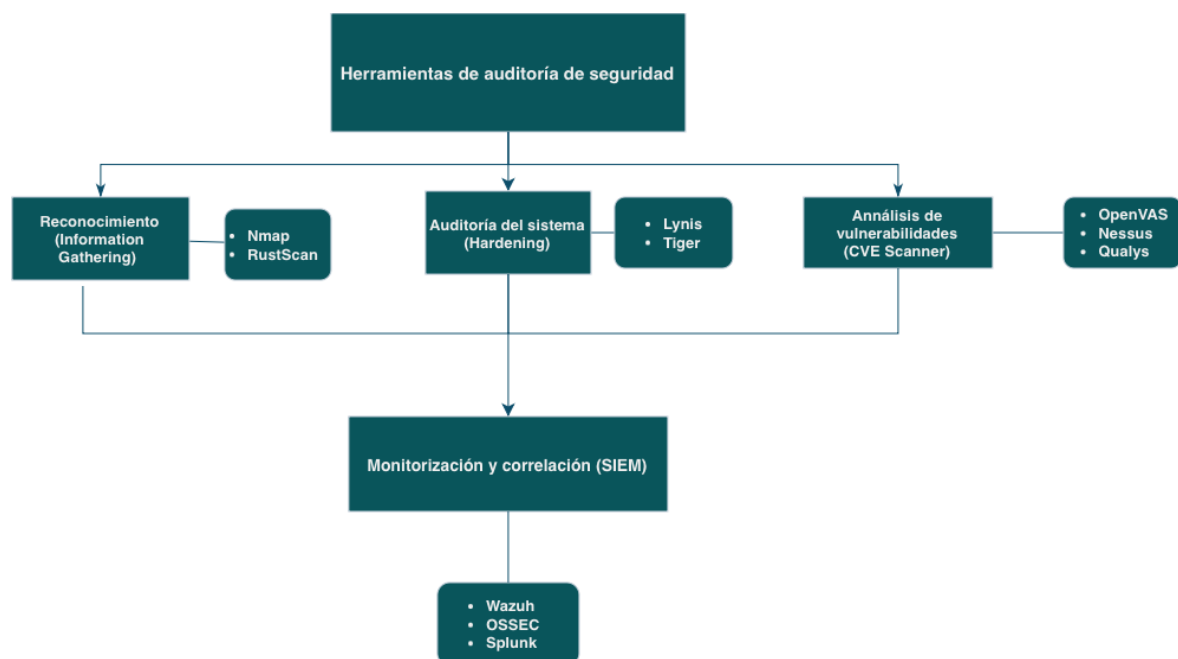


Figura 1. Clasificación funcional de las principales herramientas utilizadas durante una auditoría de seguridad en sistemas Linux. Fuente: elaboración propia.

1.3 Herramientas de reconocimiento (Information Gathering)

Las herramientas de reconocimiento tienen como finalidad recopilar información inicial sobre el sistema o la infraestructura que será objeto de análisis. Se trata de la primera fase de cualquier auditoría, ya que proporciona una visión general del entorno antes de realizar comprobaciones más profundas.

Entre la información obtenida durante esta etapa destacan la identificación de servicios disponibles, puertos abiertos, sistemas operativos, interfaces de red, usuarios existentes o configuración básica del sistema.

Una de las herramientas más conocidas dentro de esta categoría es **Nmap**, considerada el estándar de facto para el descubrimiento de equipos y servicios en redes TCP/IP. Nmap permite identificar hosts activos, detectar puertos abiertos, identificar servicios y realizar una estimación del sistema operativo remoto mediante técnicas de *fingerprinting*. Gracias a su versatilidad, constituye una herramienta ampliamente utilizada en auditorías de red y pruebas de penetración.

Otra solución ampliamente utilizada es **RustScan**, diseñada para acelerar el proceso de detección de puertos abiertos mediante un motor desarrollado en Rust. Su principal ventaja reside en la elevada velocidad de escaneo, aunque normalmente complementa su funcionamiento utilizando Nmap para realizar un análisis más detallado de los resultados obtenidos.

A pesar de su utilidad, estas herramientas presentan una limitación importante cuando el objetivo consiste en analizar el propio sistema Linux desde el interior. Su finalidad principal es descubrir información relacionada con la infraestructura de red y los servicios accesibles, pero no realizan una recopilación detallada de la configuración interna del sistema operativo.

1.4 Herramientas de auditoría del sistema

Una segunda categoría está formada por aquellas herramientas cuyo objetivo consiste en analizar la configuración interna del sistema operativo para detectar configuraciones inseguras, servicios innecesarios, permisos incorrectos o incumplimientos de buenas prácticas de seguridad.

Dentro de este grupo destaca **Lynis**, una de las herramientas de auditoría para sistemas Linux más utilizadas en entornos profesionales. Lynis realiza

numerosas comprobaciones relacionadas con la configuración del kernel, permisos de archivos, políticas de autenticación, servicios instalados, configuración de red, mecanismos de protección y recomendaciones de *hardening*. Como resultado genera un informe detallado acompañado de sugerencias para mejorar la seguridad del sistema.

Otra herramienta representativa es **Tiger**, una solución clásica orientada a detectar problemas de configuración y posibles riesgos de seguridad en sistemas UNIX y Linux. Aunque actualmente su utilización es menor que la de Lynis, sigue siendo un referente histórico dentro de las herramientas de auditoría del sistema.

Estas soluciones proporcionan un análisis muy exhaustivo del entorno, aunque generan una gran cantidad de información que requiere experiencia para ser interpretada correctamente. Además, muchas de sus comprobaciones están orientadas a administradores de sistemas y auditores con conocimientos avanzados de Linux.

1.5 Escáneres de vulnerabilidades

Mientras que las herramientas anteriores se centran en recopilar información o evaluar configuraciones, los escáneres de vulnerabilidades tienen como objetivo identificar debilidades conocidas presentes en aplicaciones, servicios o sistemas operativos.

Entre las soluciones más extendidas se encuentra **OpenVAS**, plataforma de código abierto capaz de comparar los servicios detectados con bases de datos de vulnerabilidades conocidas (CVE), generando informes que incluyen el nivel de riesgo y recomendaciones de mitigación.

Otra herramienta ampliamente implantada en el ámbito profesional es **Nessus**, desarrollada por Tenable. Gracias a su extensa base de plugins y a la actualización continua de firmas de vulnerabilidades, constituye uno de los escáneres comerciales más utilizados por empresas y consultoras de ciberseguridad.

También destaca **Qualys VMDR**, una plataforma cloud que combina análisis de vulnerabilidades, gestión de activos y evaluación continua del riesgo sobre infraestructuras distribuidas.

La principal característica de este tipo de herramientas es que no se limitan a recopilar información, sino que intentan determinar si el sistema presenta vulnerabilidades conocidas que puedan ser explotadas por un atacante.

1.6 Plataformas SIEM y monitorización continua

La última gran categoría corresponde a las plataformas SIEM (*Security Information and Event Management*), cuya finalidad consiste en recopilar, correlacionar y analizar eventos de seguridad procedentes de diferentes dispositivos y sistemas.

Entre las soluciones de código abierto más utilizadas destaca **Wazuh**, plataforma que permite monitorizar equipos mediante agentes instalados en los sistemas, analizar registros, detectar modificaciones en archivos críticos, controlar políticas de seguridad y generar alertas automáticas ante comportamientos anómalos.

Otra plataforma de referencia es **OSSEC**, precursor de Wazuh y orientado igualmente a la detección de intrusiones mediante el análisis de registros y la supervisión continua del sistema.

En el ámbito comercial sobresale **Splunk**, solución ampliamente utilizada para la centralización y análisis de grandes volúmenes de eventos de seguridad en infraestructuras empresariales.

A diferencia de las herramientas de auditoría tradicionales, las plataformas SIEM no realizan un análisis puntual, sino una supervisión continua del entorno con el objetivo de detectar incidentes de seguridad en tiempo real.

1.7 Comparativa de las soluciones existentes

Las herramientas descritas anteriormente cubren diferentes necesidades dentro del proceso de auditoría de seguridad. Sin embargo, **las soluciones analizadas no están específicamente orientadas a ofrecer una herramienta ligera destinada exclusivamente a automatizar la recopilación inicial de información local del sistema mediante una única ejecución.**

Categoría	Herramienta	Función principal	Ventajas	Limitaciones
Reconocimiento	Nmap	Descubrimiento de equipos y servicios	Muy completo y flexible	No analiza la configuración interna del sistema
Reconocimiento	RustScan	Escaneo rápido de puertos	Gran velocidad	Funcionalidad limitada al reconocimiento de red
Auditoría	Lynis	Auditoría de seguridad en Linux	Muy completo	Genera informes extensos y requiere experiencia

Categoría	Herramienta	Función principal	Ventajas	Limitaciones
Auditoría	Tiger	Auditoría del sistema	Ligero y sencillo	Menor desarrollo y utilización en la actualidad
Vulnerabilidades	OpenVAS	Detección de vulnerabilidades	Base de datos CVE	Instalación y mantenimiento complejos
Vulnerabilidades	Nessus	Evaluación de vulnerabilidades	Gran precisión	Licencia comercial
SIEM	Wazuh	Monitorización continua	Amplias capacidades de detección	Arquitectura compleja
SIEM	Splunk	Gestión de eventos	Alta escalabilidad	Coste elevado
Información Gathering	LinuxRecon	Recopilación automatizada y síntesis inicial de información del sistema	Ligero, modular, ejecución controlada y generación centralizada de resultados	No realiza detección automática de vulnerabilidades

Tabla 1. Comparativa de herramientas de ciberseguridad y posicionamiento de LinuxRecon. Fuente: elaboración propia.

1.8 Posicionamiento de LinuxRecon

Tras analizar las principales soluciones existentes puede observarse que la mayoría de las herramientas estudiadas están orientadas a realizar auditorías completas, escaneos de vulnerabilidades o monitorización continua de infraestructuras. Estas soluciones ofrecen un elevado número de funcionalidades, aunque normalmente requieren configuraciones más complejas, un mayor consumo de recursos o conocimientos especializados para interpretar correctamente los resultados obtenidos.

LinuxRecon se sitúa en un nivel diferente dentro del proceso de auditoría. Su objetivo no consiste en sustituir herramientas consolidadas como Nmap, Lynis, OpenVAS o Wazuh, sino en complementar el proceso de evaluación de seguridad mediante la automatización de la fase inicial de recopilación de información del sistema.

La herramienta recopila de forma automática información general del sistema y de la distribución Linux, identidad y cuentas de usuario, alias configurados, procesos activos, puertos y sockets en escucha, interfaces y rutas de red, puntos de montaje, utilización de los sistemas de archivos, archivos con permisos SUID y eventos recientes registrados por el sistema. Los resultados obtenidos se centralizan en un único informe de texto estructurado, denominado `report.txt`, facilitando su consulta y posterior análisis.

Además de presentar los datos recopilados, LinuxRecon incorpora al final del informe un **resumen de resultados orientado a la revisión de seguridad**, que sintetiza algunos de los principales elementos obtenidos durante la ejecución. Este resumen permite identificar rápidamente información relevante del sistema, como la distribución y arquitectura detectadas, el número de sockets identificados, la cantidad de archivos con permisos SUID y los eventos de tipo *warning* recopilados. Asimismo, señala determinados aspectos que pueden resultar de interés para una revisión posterior por parte del analista.

Este mecanismo no constituye un sistema automático de detección de vulnerabilidades. La presencia de un archivo con permisos SUID, un puerto o socket en escucha o un evento de tipo *warning* no implica por sí misma la existencia de una vulnerabilidad. Estos elementos representan información potencialmente relevante que debe ser contrastada con la configuración esperada del sistema y evaluada posteriormente por el profesional encargado de la auditoría.

La aproximación adoptada presenta varias ventajas. En primer lugar, reduce la necesidad de ejecutar individualmente múltiples comandos durante una recopilación inicial de información. En segundo lugar, centraliza los resultados en un único informe estructurado, facilitando su consulta, conservación y revisión posterior. Finalmente, la organización funcional del código facilita futuras ampliaciones mediante la incorporación de nuevas funciones de recopilación sin alterar el objetivo principal de la herramienta.

En consecuencia, LinuxRecon puede considerarse una solución de apoyo a la fase de *Information Gathering*, especialmente adecuada para auditorías básicas, actividades formativas y evaluaciones preliminares de sistemas Linux. Su finalidad es proporcionar una visión inicial, organizada y fácilmente interpretable del entorno analizado antes de utilizar herramientas especializadas para realizar fases posteriores de evaluación de seguridad.

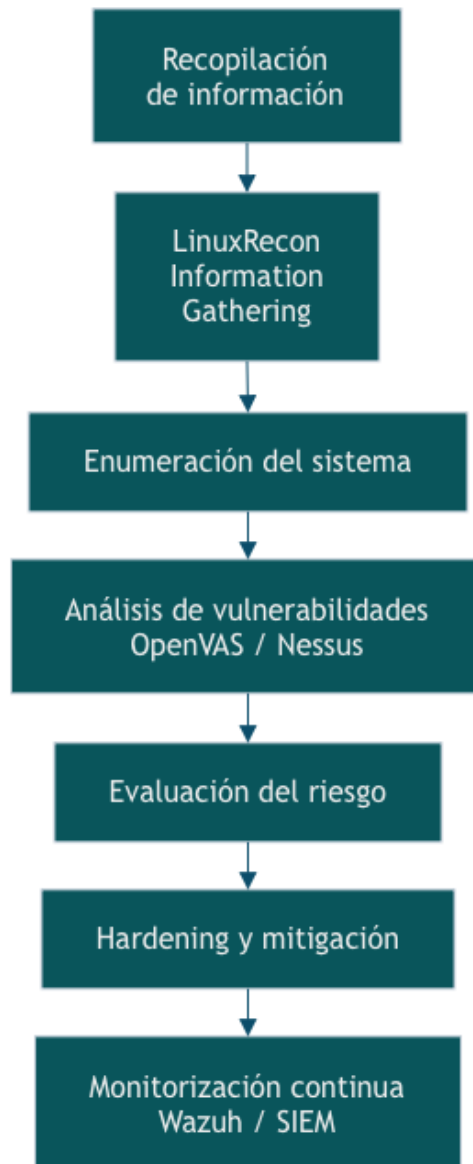


Figura 2. Posicionamiento de LinuxRecon dentro del ciclo de una auditoría de seguridad. Fuente: elaboración propia.

2. MOTIVACIÓN

La transformación digital experimentada por las organizaciones durante los últimos años ha incrementado considerablemente la superficie de exposición frente a amenazas cibernéticas. La consolidación de servicios en la nube, la virtualización de infraestructuras, la adopción de arquitecturas híbridas y el crecimiento de dispositivos conectados han convertido la seguridad de los sistemas Linux en un aspecto crítico para empresas, administraciones públicas y proveedores de servicios. Debido a su estabilidad, flexibilidad y elevado rendimiento, Linux se encuentra ampliamente implantado en servidores, centros de datos, infraestructuras *cloud* y numerosos dispositivos IoT, lo que incrementa la importancia de disponer de mecanismos que permitan conocer y revisar adecuadamente la configuración de estos sistemas.

Esta situación refuerza la necesidad de realizar auditorías preventivas que permitan conocer el estado de los sistemas y detectar elementos que requieran una revisión posterior. En este sentido, los informes recientes sobre ciberseguridad muestran la relevancia que continúa teniendo la explotación de vulnerabilidades como mecanismo de acceso inicial. El **Verizon Data Breach Investigations Report (DBIR) 2026** señala que la explotación de vulnerabilidades representa el **31 % de los accesos iniciales en las brechas analizadas**, situándose por primera vez por encima del abuso de credenciales como principal vía de entrada. El mismo informe señala además las dificultades existentes para remediar con rapidez vulnerabilidades críticas, reforzando la necesidad de mejorar la visibilidad y el conocimiento de los sistemas que deben protegerse.

De forma complementaria, el **IBM X-Force Threat Intelligence Index 2026** identifica un incremento interanual del **44 % en los ataques iniciados mediante la explotación de aplicaciones expuestas públicamente**. IBM relaciona esta tendencia, entre otros factores, con deficiencias en controles básicos de seguridad y con una superficie de ataque cada vez más amplia y compleja. Estos resultados ponen de manifiesto que, incluso ante la evolución de las técnicas utilizadas por los atacantes, mantener una adecuada visibilidad sobre los sistemas y aplicar controles fundamentales continúa siendo una necesidad prioritaria.

En este contexto, la fase inicial de **Information Gathering** adquiere una importancia fundamental dentro de un proceso de auditoría. Antes de realizar un análisis de vulnerabilidades o aplicar medidas de endurecimiento (*hardening*), resulta necesario conocer las características y

el estado del entorno analizado. Esto implica recopilar información relacionada con el sistema operativo y su distribución, arquitectura, usuarios e identidad, procesos activos, puertos y sockets en escucha, interfaces y rutas de red, puntos de montaje, utilización de los sistemas de archivos, permisos especiales y determinados eventos registrados por el sistema.

Una recopilación incompleta o desorganizada puede dificultar el trabajo posterior del analista. Además, cuando esta fase se realiza manualmente, puede ser necesario ejecutar sucesivamente diferentes comandos, consultar distintos recursos del sistema y revisar numerosas salidas de terminal. Este procedimiento incrementa la posibilidad de omitir información, dificulta la repetibilidad del proceso y provoca que los resultados permanezcan dispersos, complicando su posterior consulta y análisis.

Actualmente existen herramientas consolidadas como **Nmap**, **Lynis**, **OpenVAS** o **Wazuh**, capaces de cubrir diferentes fases de una auditoría, desde el reconocimiento de red hasta el análisis de vulnerabilidades o la monitorización continua. Sin embargo, estas soluciones responden a objetivos más amplios y presentan niveles de complejidad diferentes. En determinados escenarios, como auditorías preliminares, laboratorios de ciberseguridad, actividades formativas o evaluaciones iniciales de sistemas Linux, puede resultar útil disponer de una herramienta más sencilla cuyo propósito se limite a automatizar y centralizar la recopilación inicial de información.

La motivación principal de este trabajo surge precisamente de esta necesidad. Se plantea el desarrollo de **LinuxRecon**, una herramienta ligera escrita en Python destinada a automatizar diferentes operaciones de recopilación de información sobre un sistema Linux y centralizar los resultados obtenidos en un único informe estructurado.

La versión desarrollada permite recopilar información general del sistema y de la distribución utilizada, identidad y cuentas de usuario, alias, procesos activos, puertos y sockets en escucha, interfaces y rutas de red, puntos de montaje, utilización de los sistemas de archivos, archivos con permisos SUID y determinados registros recientes del sistema. Toda esta información se integra automáticamente en el archivo `report.txt`, proporcionando al analista una fuente centralizada para su posterior revisión.

Además de centralizar los datos obtenidos, LinuxRecon incorpora un **resumen de resultados orientado a la revisión de seguridad**. Este resumen sintetiza determinados elementos obtenidos durante la ejecución, como la distribución y arquitectura identificadas, el número de sockets

detectados, la cantidad de archivos con permisos SUID y los eventos de tipo *warning* recopilados. Su finalidad consiste en facilitar una primera lectura de los resultados y señalar información que pueda requerir una comprobación posterior.

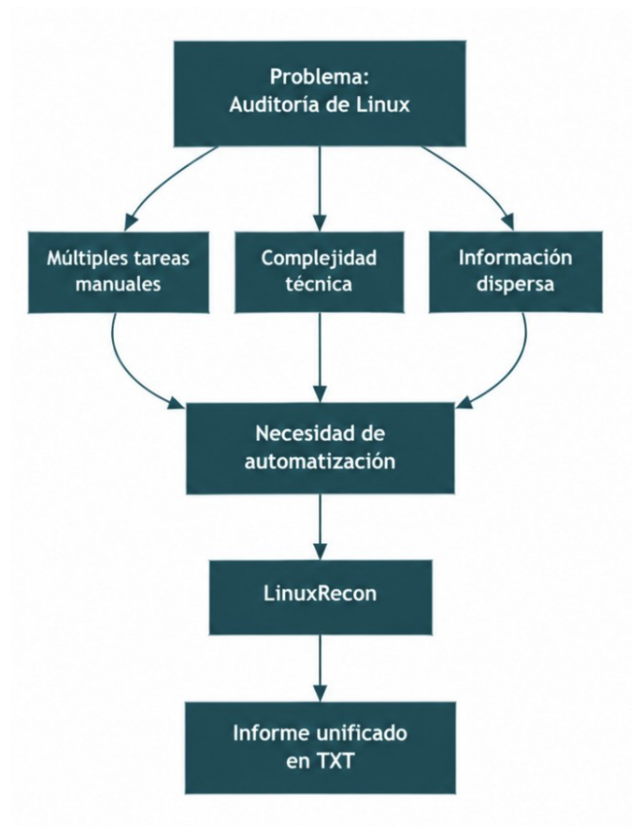
Este mecanismo no pretende convertir LinuxRecon en un escáner de vulnerabilidades. La herramienta no determina automáticamente que un elemento detectado constituya una vulnerabilidad, sino que recopila, organiza y sintetiza información relevante para que posteriormente pueda ser interpretada por un analista o utilizada como punto de partida para herramientas especializadas.

Otro aspecto considerado durante el desarrollo es la **seguridad de la propia ejecución de la herramienta**. Al tratarse de una aplicación que interactúa con recursos y comandos del sistema operativo, resulta necesario reducir los riesgos asociados a la ejecución de subprocessos. Por este motivo, la implementación final utiliza una ejecución controlada de comandos mediante listas de argumentos, evitando la utilización de `shell=True`, incorpora tiempos máximos de ejecución (*timeouts*) y contempla el tratamiento de diferentes condiciones de error. Asimismo, el informe generado se protege mediante permisos `0600`, restringiendo su lectura y modificación al usuario propietario, dado que puede contener información potencialmente sensible sobre el sistema analizado.

La aproximación adoptada no pretende sustituir plataformas profesionales de auditoría, monitorización o análisis de vulnerabilidades. LinuxRecon se plantea como una herramienta complementaria destinada específicamente a la fase inicial de *Information Gathering*, proporcionando un punto de partida homogéneo y reutilizable para posteriores procesos de evaluación de seguridad. Su automatización reduce la necesidad de ejecutar manualmente múltiples operaciones y permite conservar los resultados de forma estructurada para su posterior consulta.

Desde una perspectiva académica, el desarrollo de LinuxRecon permite aplicar de forma práctica conocimientos relacionados con administración de sistemas Linux, automatización mediante Python, gestión segura de subprocessos, tratamiento de errores, permisos del sistema de archivos, recopilación y estructuración de información y diseño funcional de herramientas de ciberseguridad. Asimismo, permite abordar el ciclo completo de desarrollo de una solución técnica: identificación de una necesidad, diseño, implementación, aplicación de medidas de seguridad, ejecución de pruebas y validación de los resultados obtenidos.

En consecuencia, la motivación del proyecto no reside únicamente en automatizar una serie de comandos, sino en desarrollar una solución reproducible y controlada que permita **recopilar, centralizar y sintetizar información relevante de un sistema Linux**, reduciendo la dispersión de los resultados y proporcionando una base organizada para posteriores



tareas de análisis y auditoría.

Figura 3. Motivación del desarrollo de LinuxRecon y problema que resuelve. Fuente: elaboración propia.

3. HIPÓTESIS

3.1 Planteamiento de la hipótesis

Tras analizar el estado actual de las herramientas de auditoría y justificar la necesidad de optimizar la fase inicial de recopilación de información, se plantea la siguiente hipótesis de investigación:

La automatización de la fase de *Information Gathering* mediante una herramienta ligera desarrollada en Python permite centralizar y homogeneizar la obtención de información relevante de un sistema Linux, reducir la necesidad de ejecutar manualmente múltiples operaciones y disminuir el riesgo de omisiones durante la recopilación, proporcionando una base estructurada y reproducible para posteriores tareas de auditoría de seguridad.

Esta hipótesis parte de la consideración de que una parte importante del trabajo realizado durante las fases iniciales de una auditoría consiste en ejecutar diferentes operaciones sobre el sistema operativo con el objetivo de conocer las características y el estado de la máquina analizada.

Habitualmente, el auditor debe consultar de forma independiente información relativa al sistema operativo y su distribución, identidad y cuentas de usuario, procesos en ejecución, puertos y sockets en escucha, interfaces y rutas de red, sistemas de archivos, permisos especiales y determinados registros del sistema.

Aunque muchas de estas operaciones pueden realizarse mediante comandos nativos de Linux, su ejecución individual obliga al analista a repetir una secuencia de acciones, consultar distintas fuentes y organizar posteriormente los resultados obtenidos. Esta dispersión puede dificultar la repetibilidad del procedimiento y aumentar la posibilidad de omitir información relevante durante la fase inicial de reconocimiento.

Se plantea que este procedimiento puede automatizarse mediante una única aplicación capaz de realizar de forma controlada las operaciones de recopilación, almacenar sus resultados y presentarlos mediante una estructura homogénea. De esta forma, diferentes ejecuciones pueden seguir el mismo procedimiento y producir un informe organizado que pueda conservarse y utilizarse posteriormente como punto de partida para otras tareas de evaluación de seguridad.

La hipótesis no presupone que la automatización permita determinar automáticamente la existencia de vulnerabilidades. LinuxRecon se plantea como una herramienta de apoyo al analista: recopila, organiza y sintetiza información potencialmente relevante, mientras que la interpretación de los resultados y la determinación del riesgo permanecen fuera del alcance de la aplicación.

3.2 Propuesta de solución

Con el objetivo de comprobar la hipótesis planteada, se desarrolla **LinuxRecon**, una herramienta de línea de comandos orientada a automatizar y estructurar la fase inicial de *Information Gathering* en sistemas Linux.

La herramienta está implementada en Python y combina la consulta de recursos del propio sistema con la ejecución controlada de utilidades nativas de Linux. La versión final incorpora medidas destinadas a reducir los riesgos derivados de la ejecución de subprocesos. Los comandos se proporcionan mediante listas de argumentos y se ejecutan sin utilizar `shell=True`, evitando que sean interpretados por un intérprete de comandos. Asimismo, se establecen tiempos máximos de ejecución (*timeouts*) y se contempla el tratamiento de diferentes condiciones de error.

LinuxRecon recopila información general del sistema, kernel, arquitectura y distribución; identidad y cuentas de usuario; UID, GID y grupos; alias configurados; procesos activos; puertos y sockets en escucha; interfaces y rutas de red; puntos de montaje; utilización de los sistemas de archivos; archivos con permisos SUID; y determinados registros y eventos recientes del sistema.

Los resultados obtenidos se centralizan automáticamente en un único archivo denominado `report.txt`. Este informe presenta la información mediante secciones diferenciadas y contiene además un **resumen de resultados orientado a la revisión de seguridad**, destinado a sintetizar algunos de los principales datos obtenidos durante la ejecución y señalar elementos que puedan resultar de interés para una revisión posterior.

Debido a que el informe puede contener información potencialmente sensible sobre el sistema analizado, LinuxRecon aplica permisos `0600` a `report.txt`, restringiendo su lectura y modificación al usuario propietario.

El diseño conserva deliberadamente un alcance limitado. LinuxRecon no realiza detección automática de vulnerabilidades, no asigna niveles de riesgo a los elementos identificados y no pretende sustituir herramientas especializadas de auditoría o análisis de vulnerabilidades. Su aportación consiste en proporcionar una recopilación local automatizada, homogénea, controlada y reproducible que pueda utilizarse como base para análisis posteriores.

3.3 Objetivos del proyecto

3.3.1 Objetivo general

El objetivo general del presente Trabajo Fin de Máster consiste en **diseñar, desarrollar y validar LinuxRecon, una herramienta de ciberseguridad capaz de automatizar de forma controlada la recopilación inicial de información relevante de un sistema Linux, centralizando y estructurando los resultados obtenidos y aplicando medidas básicas de seguridad sobre la ejecución de subprocesos y la protección del informe generado.**

La herramienta se orienta específicamente a la fase de *Information Gathering*. No pretende sustituir escáneres de vulnerabilidades ni plataformas profesionales de auditoría, sino proporcionar una solución ligera y reproducible que ofrezca una primera visión organizada del sistema y una base reutilizable para posteriores tareas de evaluación de seguridad.

3.3.2 Objetivos específicos

Para alcanzar el objetivo general y comprobar la hipótesis planteada se establecen los siguientes objetivos específicos:

- Automatizar la obtención de información general del sistema, incluyendo kernel, arquitectura, distribución y versión.
- Identificar el usuario actual, UID, GID, grupos y cuentas registradas en el sistema.
- Recopilar información relativa a los alias configurados para el usuario.
- Enumerar los procesos activos y los puertos y sockets en escucha.
- Recopilar información sobre interfaces y rutas de red.
- Obtener información sobre los puntos de montaje y la utilización de los sistemas de archivos.
- Localizar archivos que presenten activado el permiso SUID como información relevante para posteriores comprobaciones de seguridad.

- Recopilar una muestra acotada de registros recientes y eventos de tipo *warning* disponibles en el sistema.
- Centralizar automáticamente los resultados obtenidos en un único archivo estructurado denominado `report.txt`.
- Generar un resumen final que sintetice determinados resultados y facilite la identificación de elementos que puedan requerir una revisión posterior.
- Aplicar medidas de seguridad a la ejecución de subprocessos mediante listas de argumentos, eliminación del uso de `shell=True`, establecimiento de tiempos máximos de ejecución y tratamiento controlado de errores.
- Proteger el informe generado mediante permisos `0600`, limitando su lectura y modificación al usuario propietario.
- Proporcionar una interfaz de terminal sencilla que permita centralizar la ejecución de las operaciones de recopilación.
- Mantener una organización funcional del código que facilite su comprensión, mantenimiento y posible ampliación futura.
- Validar experimentalmente las funcionalidades implementadas mediante la ejecución de LinuxRecon y el contraste de determinados resultados con comandos nativos equivalentes del sistema operativo.
- Comprobar el comportamiento de la aplicación ante entradas no válidas, la finalización mediante la opción correspondiente del menú, la regeneración del informe y la corrección sintáctica del código desarrollado.

Estos objetivos establecen una relación directa entre el problema identificado, la hipótesis formulada, el desarrollo de LinuxRecon y las pruebas utilizadas posteriormente para comprobar el funcionamiento de la solución.

3.4 Alcance de la hipótesis

La hipótesis planteada no pretende demostrar que LinuxRecon pueda sustituir herramientas profesionales como **Lynis**, **OpenVAS**, **Nessus** o **Wazuh**. Estas soluciones proporcionan capacidades considerablemente más amplias relacionadas con auditorías exhaustivas, detección de vulnerabilidades, monitorización continua o gestión integral de eventos de seguridad.

El alcance del presente trabajo se limita a evaluar la utilidad de automatizar y centralizar determinadas operaciones correspondientes a la fase inicial de *Information Gathering* en un sistema Linux. En consecuencia, la validación se centra en comprobar que las funcionalidades implementadas se ejecutan

correctamente, que el informe se genera y protege adecuadamente y que determinados resultados obtenidos por LinuxRecon son consistentes con los producidos mediante comandos nativos equivalentes.

La evaluación contempla asimismo aspectos relacionados con la robustez básica de la aplicación, como el tratamiento de entradas no válidas, la finalización controlada, la regeneración del informe y la comprobación sintáctica del código.

Por tanto, **no forma parte del alcance de la hipótesis demostrar una reducción cuantitativa del tiempo frente a un procedimiento manual**, puesto que para ello sería necesario diseñar un experimento comparativo específico con múltiples ejecuciones y condiciones controladas. La automatización sí permite reducir la necesidad de ejecutar individualmente múltiples operaciones y proporciona un procedimiento homogéneo y repetible, aspectos que pueden comprobarse directamente mediante la implementación y las pruebas realizadas.

Del mismo modo, los elementos destacados por LinuxRecon no deben interpretarse automáticamente como vulnerabilidades. Un archivo SUID, un socket en escucha o un evento de tipo *warning* puede formar parte del funcionamiento normal del sistema. La herramienta proporciona información para su posterior evaluación, mientras que la determinación de su relevancia desde el punto de vista de la seguridad corresponde al analista.

La validación de la hipótesis se desarrolla en los capítulos posteriores mediante pruebas funcionales y comparativas sobre el entorno experimental utilizado. Los resultados obtenidos permitirán determinar si LinuxRecon cumple los objetivos establecidos y si proporciona una recopilación inicial de información estructurada, reproducible y



técnicamente consistente.

Figura 4. Hipótesis de investigación y propuesta de solución mediante LinuxRecon.
Fuente: elaboración propia.

4. TESIS

4.1 Propuesta de la solución

Tras analizar las principales herramientas existentes para la auditoría de sistemas Linux y justificar la necesidad de optimizar la fase inicial de recopilación de información, se propone el desarrollo de **LinuxRecon**, una herramienta de línea de comandos diseñada para automatizar y estructurar el proceso de *Information Gathering* durante las primeras etapas de una auditoría de seguridad.

La propuesta parte de la premisa de que una parte significativa del trabajo inicial realizado por un analista de ciberseguridad consiste en obtener información del sistema antes de iniciar procesos más avanzados de evaluación de vulnerabilidades, análisis de riesgos o endurecimiento (*hardening*). En un procedimiento manual, esta tarea requiere ejecutar diferentes comandos, consultar archivos del sistema, revisar sus resultados y organizar posteriormente la información obtenida.

Aunque cada una de estas operaciones puede realizarse mediante utilidades nativas de GNU/Linux, su ejecución independiente implica tareas repetitivas, genera información distribuida entre diferentes salidas de terminal y aumenta la posibilidad de omitir determinadas comprobaciones.

LinuxRecon pretende simplificar este proceso mediante una única aplicación capaz de coordinar automáticamente las operaciones de recopilación y generar un informe estructurado con los resultados.

La herramienta ha sido desarrollada en **Python 3** y combina la ejecución controlada de utilidades nativas de Linux con la lectura de determinadas fuentes de información del sistema. Su funcionamiento se orienta exclusivamente a tareas de consulta y enumeración, evitando realizar modificaciones sobre la configuración del equipo analizado.

La versión final permite recopilar información relacionada con:

- sistema operativo, distribución, kernel y arquitectura;
- identidad del usuario, UID, GID y grupos;
- cuentas registradas;
- procesos activos;
- sockets TCP y UDP en escucha;
- interfaces y rutas de red;
- alias del usuario;

- puntos de montaje y utilización del almacenamiento;
- archivos con permiso SUID;
- eventos recientes y mensajes de tipo *warning*;
- síntesis informativa de determinados elementos que pueden requerir revisión.

Los resultados obtenidos se centralizan automáticamente en un único archivo denominado **report.txt**, organizado mediante secciones identificables y protegido con permisos 0600, restringiendo su lectura y modificación al propietario del archivo.

La versión final incorpora asimismo medidas destinadas a reforzar la ejecución de las operaciones del sistema. Los comandos externos se ejecutan mediante argumentos estructurados y sin utilizar un intérprete de *shell*, reduciendo la superficie asociada a la interpretación de comandos. El mecanismo común de ejecución incorpora además límites temporales (*timeouts*) y tratamiento diferenciado de determinadas condiciones de error.

LinuxRecon no pretende sustituir plataformas especializadas como Lynis, OpenVAS, Nessus o Wazuh. Su ámbito se sitúa en una etapa anterior del proceso: proporcionar al analista una primera recopilación local, homogénea y reproducible que pueda utilizarse posteriormente como apoyo para análisis más avanzados.

El diseño se sustenta en cinco principios:

Automatización. Las distintas operaciones necesarias para recopilar información se coordinan desde una única ejecución.

Simplicidad. La herramienta proporciona una interfaz de terminal reducida y comprensible.

Modularidad lógica. Aunque la implementación actual permanece concentrada principalmente en `my.py`, las responsabilidades están separadas mediante funciones especializadas.

Seguridad de ejecución. Las operaciones externas se realizan de forma controlada, sin *shell*, con argumentos predefinidos, tratamiento de errores y límites temporales.

Protección de resultados. El informe generado recibe permisos restrictivos debido al carácter potencialmente sensible de la información recopilada.

De esta forma, LinuxRecon constituye una herramienta de apoyo a la fase inicial de auditoría cuyo objetivo no es determinar automáticamente la

existencia de vulnerabilidades, sino proporcionar información organizada



que facilite posteriores decisiones del analista.

Figura 5. Propuesta de valor de LinuxRecon como herramienta de apoyo a la recopilación inicial. Fuente: elaboración propia.

4.2 Objetivos funcionales

Los objetivos funcionales describen las capacidades concretas implementadas en LinuxRecon y permiten establecer posteriormente una correspondencia directa entre requisitos, componentes y pruebas.

OF-1. Obtener información general del sistema

LinuxRecon debe identificar automáticamente información relacionada con la distribución GNU/Linux, versión, kernel y arquitectura del sistema analizado.

OF-2. Identificar el contexto de ejecución

La herramienta debe identificar el usuario que realiza la ejecución, así como su UID, GID y grupos asociados.

OF-3. Enumerar cuentas y alias

LinuxRecon debe recuperar las cuentas registradas en el sistema y recopilar los alias disponibles para el usuario actual cuando sea posible.

OF-4. Obtener los procesos en ejecución

La herramienta debe enumerar los procesos activos en el momento de realizar la recopilación.

OF-5. Identificar sockets en escucha

LinuxRecon debe obtener información sobre sockets TCP y UDP en estado de escucha, permitiendo conocer servicios potencialmente expuestos.

OF-6. Obtener información de red

La herramienta debe recopilar información relativa a interfaces, direcciones y rutas configuradas.

OF-7. Analizar información de almacenamiento

LinuxRecon debe identificar los puntos de montaje existentes y el uso de los sistemas de archivos.

OF-8. Localizar archivos SUID

La herramienta debe localizar archivos que tengan activo el permiso especial SUID y proporcionar información que permita su revisión posterior.

OF-9. Recopilar eventos recientes

LinuxRecon debe obtener una muestra acotada de registros recientes y mensajes de tipo *warning* disponibles en el sistema.

OF-10. Generar un resumen informativo de seguridad

La herramienta debe producir una síntesis de determinados elementos recopilados que puedan requerir atención, sin clasificarlos automáticamente como vulnerabilidades.

OF-11. Generar un informe estructurado y protegido

Todos los resultados deben centralizarse en `report.txt`, organizado mediante secciones y protegido mediante permisos `0600`.

OF-12. Automatizar el proceso completo

Las comprobaciones deben ejecutarse mediante un flujo centralizado iniciado desde la interfaz de terminal.

OF-13. Ejecutar las operaciones externas de forma controlada

Las operaciones ejecutadas mediante `subprocess` deben emplear listas de argumentos, evitar el intérprete de `shell`, aplicar límites temporales y gestionar las incidencias sin provocar la finalización completa de la



recopilación.

Figura 6. Relación entre los objetivos funcionales y los componentes principales de LinuxRecon. Fuente: elaboración propia.

Tabla 2. Requisitos funcionales de LinuxRecon

ID	Requisito funcional	Prioridad
RF-01	Obtener información de distribución, kernel y arquitectura	Alta
RF-02	Identificar usuario, UID, GID y grupos	Alta
RF-03	Enumerar cuentas y alias	Alta
RF-04	Enumerar procesos activos	Alta
RF-05	Identificar sockets TCP/UDP en escucha	Alta
RF-06	Obtener interfaces y rutas de red	Alta
RF-07	Obtener puntos de montaje y uso de disco	Alta
RF-08	Localizar archivos SUID	Media
RF-09	Recopilar logs recientes y <i>warnings</i>	Media
RF-10	Generar un resumen informativo	Alta
RF-11	Generar report.txt estructurado	Alta
RF-12	Proteger report.txt mediante permisos 0600	Alta
RF-13	Ejecutar comandos sin <i>shell</i> y con <i>timeout</i>	Alta
RF-14	Gestionar errores sin interrumpir toda la ejecución	Alta
RF-15	Centralizar el proceso desde el menú	Alta

Tabla 3. Correspondencia entre objetivos y componentes

Objetivo	Componente
OF-1	Recopilación de información del sistema
OF-2	Identidad y privilegios
OF-3	Cuentas y alias
OF-4	Procesos
OF-5	Sockets

Objetivo	Componente
OF-6	Red
OF-7	Almacenamiento
OF-8	Permisos SUID
OF-9	Eventos y <i>warnings</i>
OF-10	Generador de resumen
OF-11	Generador y protección del informe
OF-12	Coordinador de ejecución
OF-13	<code>run_command()</code>

La ampliación de estos objetivos respecto al prototipo inicial resulta necesaria porque la versión actual ya incorpora almacenamiento, eventos, síntesis y protección del informe.

4.3 Metodología de desarrollo

El desarrollo de LinuxRecon se ha realizado siguiendo una **metodología incremental**, basada en la incorporación progresiva de funcionalidades y en su validación antes de integrarlas en el flujo completo.

Este enfoque resulta adecuado para LinuxRecon debido a que las diferentes operaciones de recopilación presentan responsabilidades diferenciadas y pueden desarrollarse y comprobarse individualmente.

4.3.1 Fases de desarrollo

El proceso se estructuró en seis fases.

Fase 1. Investigación y análisis.

Se estudió el proceso de auditoría inicial de sistemas Linux y las capacidades de herramientas existentes como Nmap, Lynis, OpenVAS y Wazuh.

Fase 2. Diseño.

Se definieron el alcance, los requisitos, las funciones de recopilación, el flujo de ejecución y la estructura inicial del informe.

Fase 3. Desarrollo inicial.

Se implementó en Python una primera versión funcional capaz de recopilar información básica del sistema.

Fase 4. Ampliación y hardening.

Se incorporaron nuevas fuentes de información —alias, almacenamiento, logs y *warnings*—, el resumen informativo y las medidas de protección de la ejecución y del informe.

Fase 5. Integración y validación.

Los componentes fueron integrados en el flujo completo y contrastados mediante ejecuciones reales y comandos nativos equivalentes.

Fase 6. Documentación.

Se elaboraron la memoria, diagramas, evidencias, documentación de utilización, presentación y materiales asociados a la defensa.

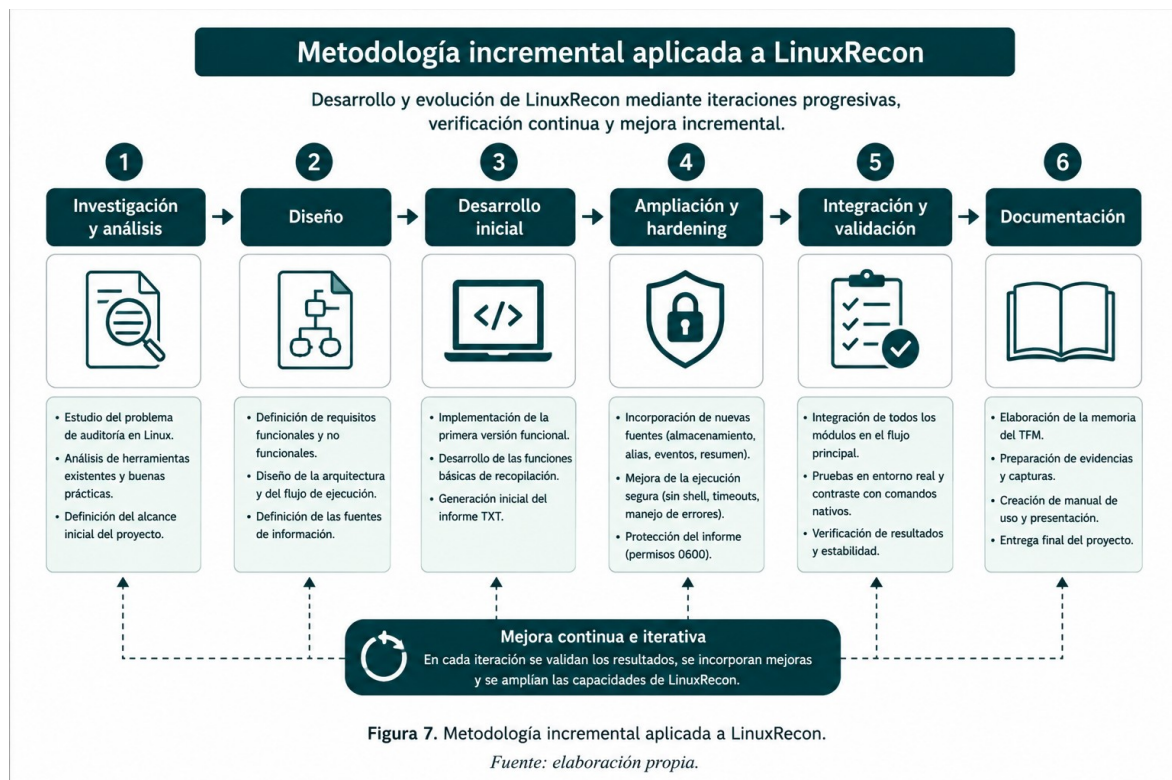


Figura 7. Metodología incremental aplicada al desarrollo de LinuxRecon. Fuente: elaboración propia.

4.3.2 Justificación

El enfoque incremental permitió disponer de una primera versión funcional y evolucionarla posteriormente sin sustituir la lógica fundamental de la aplicación.

Además, permitió comprobar cada nueva capacidad antes de incorporarla al flujo global y facilitó especialmente la fase de *hardening*, en la que se modificó el mecanismo común de ejecución y la protección del informe sin alterar el objetivo principal de LinuxRecon.

4.4 Planificación del proyecto

La planificación del proyecto siguió una secuencia coherente con la metodología incremental empleada.

Las principales fases fueron:

1. investigación del problema;
2. análisis del estado del arte;
3. definición de requisitos;
4. diseño de la solución;
5. implementación del prototipo;
6. ampliación funcional y *hardening*;
7. validación y pruebas;
8. documentación, presentación y entrega.

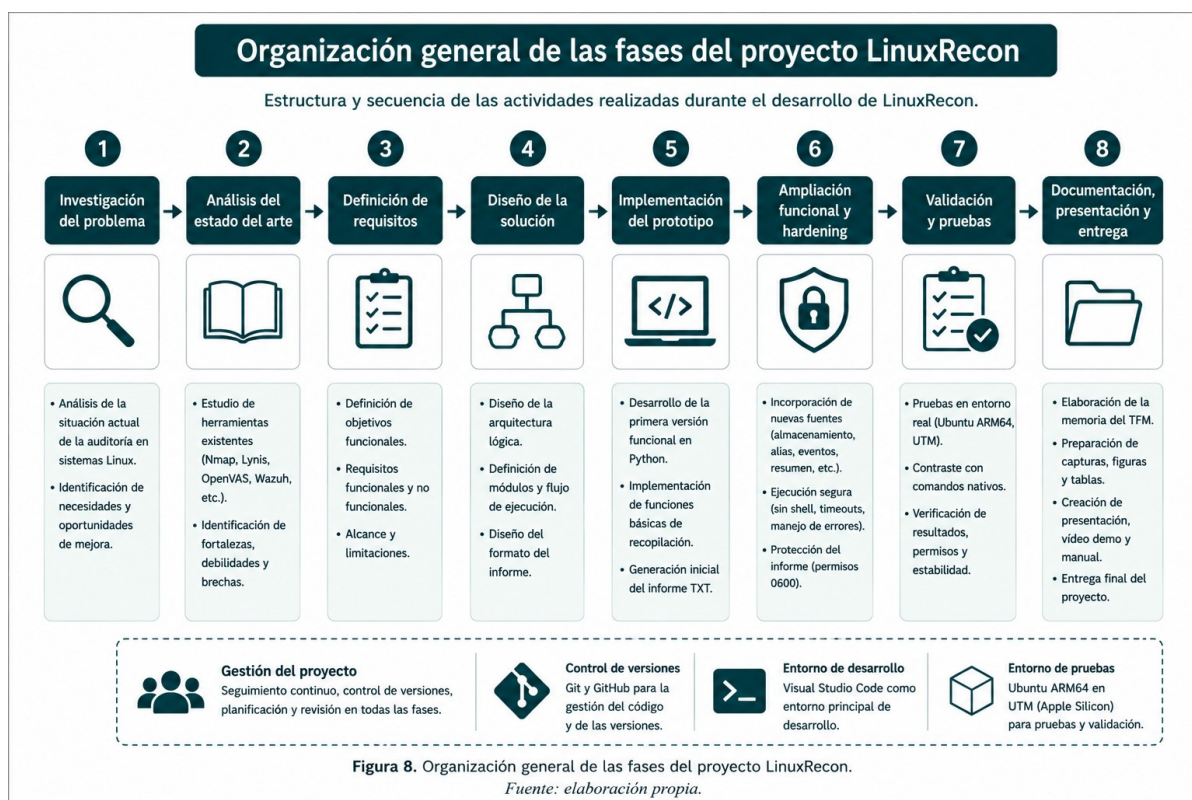
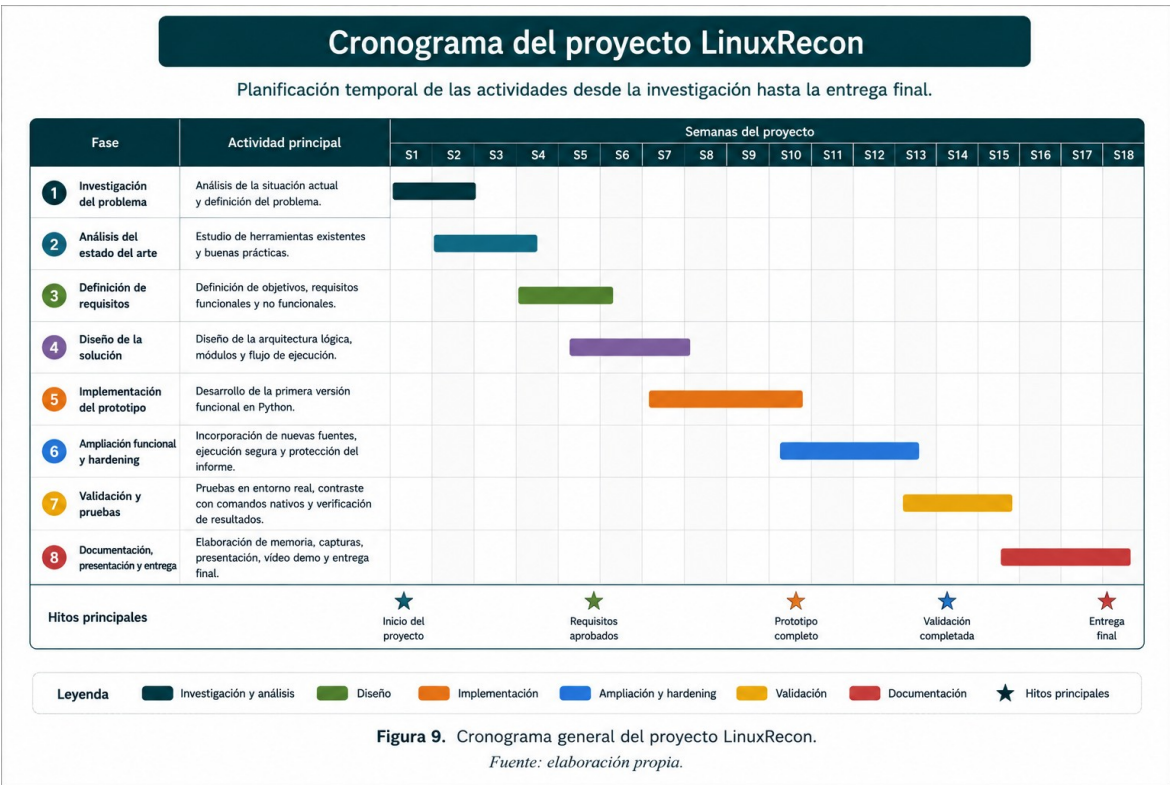


Figura 8. Organización general de las fases del proyecto LinuxRecon. Fuente: elaboración propia.

4.4.1 Cronograma

El cronograma distribuyó las actividades entre investigación, diseño,



desarrollo, integración, ampliación, pruebas y documentación.

Figura 9. Cronograma general del proyecto LinuxRecon. Fuente: elaboración propia.

4.4.2 Gestión del proyecto

El desarrollo se realizó de forma individual. Se utilizó Git para el control de versiones y GitHub como repositorio del proyecto.

El entorno de desarrollo y validación se ejecutó sobre una máquina virtual Ubuntu ARM64 mediante UTM, proporcionando un entorno aislado y reproducible.

4.5 Tecnologías utilizadas

LinuxRecon se ha desarrollado buscando minimizar dependencias externas y utilizar tecnologías ampliamente disponibles.

4.5.1 Python 3

Python constituye el lenguaje principal de desarrollo. Su biblioteca estándar permite gestionar archivos, ejecutar operaciones del sistema, procesar resultados y estructurar el informe sin introducir dependencias Python externas.

4.5.2 Ubuntu Linux

Ubuntu constituye el entorno principal utilizado para el desarrollo y validación de la herramienta.

4.5.3 UTM

UTM se emplea para ejecutar Ubuntu ARM64 sobre el equipo anfitrión Apple Silicon, proporcionando aislamiento entre el entorno de experimentación y el sistema principal.

4.5.4 Visual Studio Code

Visual Studio Code se empleó como entorno principal para la edición y mantenimiento del código.

4.5.5 Git y GitHub

Git permitió mantener el control de versiones del proyecto, mientras que GitHub se utilizó para alojar y organizar el repositorio.

4.5.6 Utilidades nativas de Linux

Tabla 4. Principales fuentes y utilidades utilizadas por LinuxRecon

Fuente/utilidad	Información obtenida
uname	Kernel, host y arquitectura
/etc/os-release	Distribución y versión
whoami / id	Usuario, UID, GID y grupos
/etc/passwd	Cuentas del sistema
ps	Procesos

Fuente/utilidad	Información obtenida
ss	Sockets TCP/UDP
ip	Interfaces y rutas
findmnt	Puntos de montaje
df	Utilización del almacenamiento
find	Archivos SUID
journalctl	Eventos recientes y warnings
información del entorno/shell	Alias del usuario

El conjunto final es más amplio que el prototipo inicial, que se centraba principalmente en sistema, usuarios, procesos, puertos, red y SUID. La versión documentada ya contempla almacenamiento y eventos, entre otras



áreas.

Figura 10. Tecnologías y fuentes de información utilizadas por LinuxRecon. Fuente: elaboración propia.

4.6 Diseño de la arquitectura

4.6.1 Arquitectura general

LinuxRecon utiliza una arquitectura local y secuencial organizada mediante funciones especializadas dentro de `my.py`.

No se utilizan bases de datos, servicios web, agentes permanentes ni componentes distribuidos.

La arquitectura puede dividirse conceptualmente en cinco niveles:

Nivel 1 — Interacción

Gestiona la comunicación con el usuario mediante el menú.

Nivel 2 — Coordinación

Controla el flujo completo de generación del informe.

Nivel 3 — Recopilación

Agrupar las funciones encargadas de obtener información del sistema.

Nivel 4 — Ejecución segura y procesamiento

Centraliza las operaciones externas, su tratamiento y la gestión de incidencias.

Nivel 5 — Síntesis y salida

Organiza los resultados, genera el resumen informativo, escribe `report.txt` y aplica los permisos restrictivos.

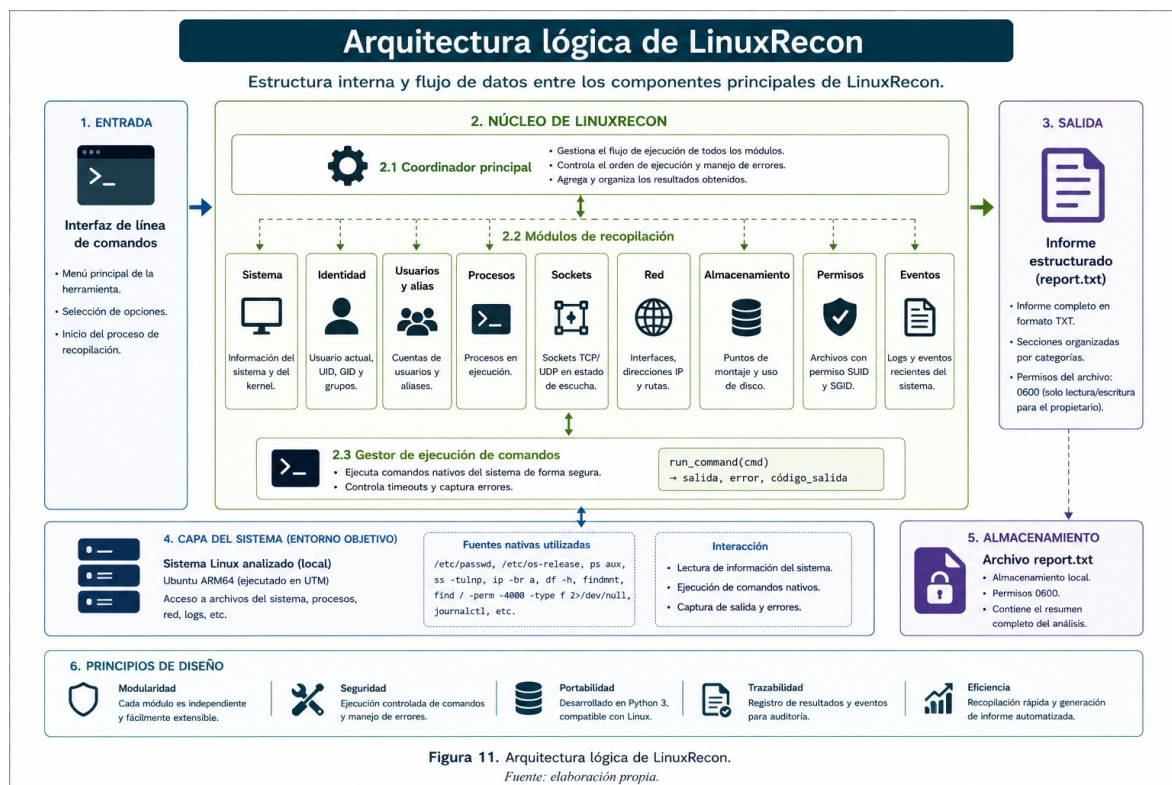


Figura 11. Arquitectura lógica de la versión final de LinuxRecon. Fuente: elaboración propia.

4.6.2 Ejecución segura

Uno de los cambios más importantes introducidos durante la evolución de LinuxRecon afecta a la ejecución de operaciones externas.

La versión final evita construir comandos que deban ser interpretados por un *shell*. En su lugar, utiliza argumentos estructurados mediante *subprocess*, lo que reduce la superficie de riesgo asociada a la interpretación de metacaracteres u operadores del intérprete.

El mecanismo común de ejecución incorpora asimismo:

- comandos predefinidos;
- ausencia de entrada libre de comandos;
- límites temporales;
- captura diferenciada de resultados e incidencias;

- continuidad de la recopilación cuando una operación concreta no puede completarse.

4.6.3 Protección del informe

`report.txt` puede contener nombres de usuario, grupos, direcciones, procesos, sockets, rutas y otros datos del sistema.

Por este motivo, la versión final aplica permisos:

```
0600
```

De esta manera, únicamente el propietario dispone de permisos de lectura y escritura.

Tabla 5. Componentes principales de LinuxRecon

Componente	Responsabilidad
<code>main()</code>	Interacción y control del menú
Coordinador del informe	Orquestación de la recopilación
<code>run_command()</code>	Ejecución controlada de operaciones
Funciones de sistema	Kernel, distribución y arquitectura
Funciones de identidad	Usuario, UID, GID, grupos y cuentas
Funciones de procesos	Procesos activos
Funciones de red	Sockets, interfaces y rutas
Funciones de almacenamiento	Montajes y utilización
Funciones de permisos	SUID
Funciones de eventos	Logs y <i>warnings</i>
Generador de resumen	Síntesis informativa
Generador de informe	Escritura y protección de <code>report.txt</code>

Tabla 6. Trazabilidad entre requisitos y arquitectura

Requisito	Componente
RF-01	Sistema
RF-02	Identidad
RF-03	Cuentas/alias

Requisito	Componente
RF-04	Procesos
RF-05	Sockets
RF-06	Red
RF-07	Almacenamiento
RF-08	SUID
RF-09	Eventos
RF-10	Síntesis
RF-11 / RF-12	Generador de informe
RF-13 / RF-14	run_command()
RF-15	Controlador

4.7 Flujo de funcionamiento

El funcionamiento de LinuxRecon sigue un flujo secuencial y controlado.

El usuario inicia la herramienta mediante:

```
python3 my.py
```

A continuación aparece el menú principal:

```
=====
LinuxRecon - TFM Tool
=====

1. Generar reporte completo
2. Salir
```

Si se selecciona la opción **1**, comienza automáticamente la recopilación.

Si se selecciona la opción **2**, la aplicación finaliza.

Las entradas diferentes de las opciones permitidas son rechazadas mediante el correspondiente mensaje de opción no válida.

Esta última circunstancia ya ha sido comprobada durante las pruebas realizadas sobre la herramienta, por lo que debe eliminarse del capítulo anterior la afirmación de que cualquier opción diferente de "1" produce directamente "Saliendo...".

El flujo completo puede representarse como:

Inicio → Menú → Validación de opción → Generación → Recopilación → Procesamiento → Resumen → Escritura de `report.txt` → Aplicación de permisos 0600 → Confirmación → Fin

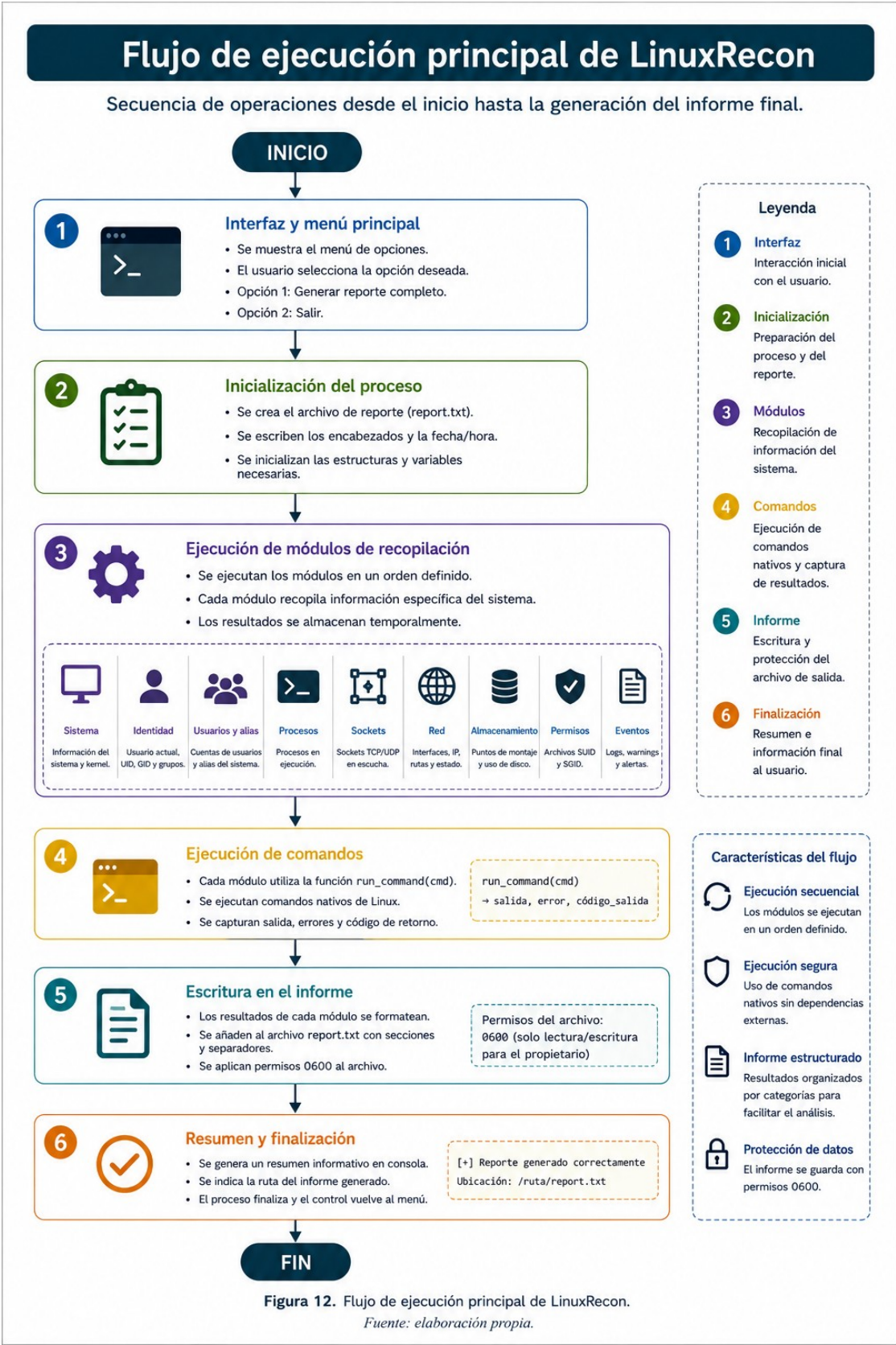


Figura 12. Flujo de ejecución principal de LinuxRecon.
Fuente: elaboración propia.

Figura 12. Flujo principal de ejecución de LinuxRecon. Fuente: elaboración propia.

4.7.1 Secuencia de generación

Durante la generación del informe se ejecutan sucesivamente las diferentes operaciones de recopilación.

Cada componente devuelve su resultado al coordinador. Los resultados son organizados mediante secciones y posteriormente utilizados para generar tanto la información detallada como el resumen informativo.

Una vez finalizada la recopilación:

1. se construye el informe;
2. se escribe `report.txt`;
3. se aplican permisos `0600`;
4. se informa al usuario de la finalización correcta.

El archivo puede comprobarse posteriormente mediante:

```
ls -l report.txt
```

y sus permisos mediante:

```
stat -c "%a %U %G %n" report.txt
```

Las pruebas realizadas han permitido comprobar un resultado equivalente a:

```
600 mire mire report.txt
```

confirmando la aplicación de la protección prevista.



Figura 13. Estructura lógica de los módulos de recopilación de LinuxRecon. Fuente: elaboración propia.

4.8 Alcance, limitaciones y plan de evolución

4.8.1 Alcance actual

LinuxRecon automatiza actualmente las siguientes áreas:

Tabla 7. Funcionalidades implementadas

Área	Funcionalidad	Estado
Sistema	Kernel, arquitectura, distribución y versión	Implementado
Identidad	Usuario, UID, GID y grupos	Implementado

Área	Funcionalidad	Estado
Usuarios	Cuentas y alias	Implementado
Procesos	Procesos activos	Implementado
Red	Interfaces y rutas	Implementado
Exposición	Sockets TCP/UDP	Implementado
Almacenamiento	Montajes y uso de disco	Implementado
Permisos	Archivos SUID	Implementado
Eventos	Logs recientes y <i>warnings</i>	Implementado
Síntesis	Resumen informativo	Implementado
Seguridad de ejecución	Argumentos estructurados, sin <i>shell</i> y <i>timeouts</i>	Implementado
Informe	report.txt y permisos 0600	Implementado
Automatización	Ejecución centralizada	Implementado

4.8.2 Funcionalidades fuera del alcance

LinuxRecon no realiza:

- explotación de vulnerabilidades;
- ataques de fuerza bruta;
- escalada automática de privilegios;
- modificación de configuraciones;
- análisis automático de CVE;
- cálculo CVSS;
- escaneo remoto;
- análisis de malware;
- monitorización continua;
- correlación SIEM;
- aplicación automática de *hardening*;
- clasificación automática de vulnerabilidades.

El resumen generado debe entenderse como **informativo**, no como un motor de detección de vulnerabilidades.

4.8.3 Limitaciones

La herramienta mantiene determinadas limitaciones.

Dependencia del entorno Linux.

Algunas operaciones dependen de utilidades disponibles en la distribución utilizada.

Dependencia de privilegios.

La información accesible depende de los permisos del usuario que ejecuta LinuxRecon.

Implementación en un único archivo.

Aunque existe modularidad lógica, gran parte de la implementación permanece concentrada en `my.py`.

Formato TXT.

El formato actual está orientado principalmente a revisión humana y no proporciona las ventajas de interoperabilidad de JSON.

Validación limitada a un entorno principal.

La validación se ha realizado fundamentalmente sobre Ubuntu ARM64, por lo que una evaluación multidistribución constituye una posible ampliación.

4.8.4 Evaluación de seguridad

Tabla 8. Seguridad de la versión final

Aspecto	Situación final	Riesgo
Modificación del sistema	Operaciones de consulta	Bajo
Entrada libre de comandos	No permitida	Bajo
Uso de <i>shell</i>	No utilizado	Bajo
Argumentos	Estructurados/predefinidos	Bajo
<i>Timeouts</i>	Implementados	Bajo
Privilegios root	No requeridos normalmente	Variable
report.txt	Permisos 0600	Reducido
Publicación del informe	Puede exponer información	Medio
Dependencias Python externas	No requeridas	Bajo

4.8.5 Evaluación de eficiencia

LinuxRecon puede considerarse **ligera por diseño**, puesto que:

- no necesita base de datos;
- no mantiene agentes residentes;
- no utiliza un servidor web;
- no requiere contenedores;
- no requiere librerías Python externas adicionales;
- utiliza recursos disponibles localmente;
- finaliza una vez generado el informe.

No obstante, la eficiencia debe demostrarse mediante mediciones y no únicamente deducirse de la arquitectura.

Tabla 9. Métricas de validación

Métrica	Método
Tiempo total	<code>time python3 my.py</code>
Tamaño del informe	<code>ls -lh report.txt</code>
Número de líneas	<code>wc -l report.txt</code>
Permisos	<code>stat -c "%a %U %G %n" report.txt</code>
Sintaxis Python	<code>python3 -m py_compile my.py</code>
Código de salida	<code>echo \$?</code>
Repetibilidad	Ejecuciones sucesivas
Correspondencia	Comparación con comandos nativos

4.8.6 Plan de evolución

La arquitectura actual permite plantear futuras ampliaciones sin modificar el objetivo fundamental del proyecto.

Nivel 1 — Mejora de usabilidad

- informes con marca temporal;
- histórico de ejecuciones;
- opciones adicionales de configuración;

- logging técnico.

Nivel 2 — Modularización física

Separación de `my.py` en paquetes especializados para sistema, usuarios, red, permisos, eventos e informes.

Nivel 3 — Nuevos módulos

Podrían añadirse comprobaciones relativas a:

- firewall;
- SSH;
- SGID;
- tareas `cron`;
- servicios;
- paquetes;
- actualizaciones;
- Docker;
- políticas de seguridad.

Nivel 4 — Nuevos formatos e integración

Podrían incorporarse formatos:

- JSON;
- HTML;

y mecanismos de integración con otras herramientas de seguridad.



Figura 14. Síntesis del estado final y evolución propuesta de LinuxRecon. Fuente: elaboración propia.

Tabla 10. Síntesis del estado final y evolución

Dimensión	Estado actual	Evolución
Funcionalidad	Recopilación local ampliada	Nuevos módulos
Arquitectura	Modularidad lógica	Paquetes/plugins
Seguridad	Ejecución endurecida	Controles adicionales
Informe	TXT + resumen + 0600	JSON/HTML
Portabilidad	Linux; validada en Ubuntu ARM64	Pruebas multidistribución
Extensibilidad	Funciones especializadas	Plugins
Integración	No implementada	APIs/SIEM
Mantenimiento	Código concentrado	Modularización física

4.8.7 Conclusión del capítulo

La tesis desarrollada en este trabajo sostiene que es posible mejorar la fase inicial de recopilación de información en sistemas Linux mediante una herramienta ligera que automatice operaciones repetitivas, centralice sus resultados y mantenga un flujo de ejecución controlado.

LinuxRecon materializa esta propuesta mediante una aplicación desarrollada en Python que recopila información del sistema, identidad, usuarios, procesos, red, sockets, almacenamiento, permisos y eventos, generando posteriormente un informe estructurado y un resumen informativo.

La evolución realizada durante el desarrollo ha permitido superar el prototipo inicial mediante la incorporación de nuevas áreas de recopilación y medidas específicas de *hardening*, entre ellas la ejecución sin intérprete de *shell*, la utilización de argumentos estructurados, los límites temporales, la gestión de errores y la protección de `report.txt` mediante permisos `0600`.

La solución mantiene deliberadamente un alcance limitado. LinuxRecon no determina automáticamente vulnerabilidades ni sustituye herramientas especializadas, sino que proporciona una **base de reconocimiento local homogénea, reproducible y protegida** sobre la que puede continuar el proceso de auditoría.

La arquitectura actual resulta suficiente para demostrar la propuesta planteada en este Trabajo Fin de Máster y constituye, al mismo tiempo, una base susceptible de evolucionar hacia una organización física modular, nuevos formatos de informe y componentes adicionales de recopilación.

5. DEMOSTRACIÓN

5.1 Implementación

5.1.1 Introducción

Una vez definida la arquitectura, los requisitos funcionales y el flujo de funcionamiento de LinuxRecon, se procedió a la implementación de la solución. El objetivo de esta fase fue transformar el diseño propuesto en una herramienta funcional capaz de ejecutarse sobre un sistema Linux real.

LinuxRecon fue desarrollada en Python 3 y validada sobre una máquina virtual Ubuntu Linux ARM64 ejecutada mediante UTM. El código fuente principal se concentra en el archivo `my.py`, manteniendo una separación lógica mediante funciones especializadas para la interacción con el usuario, la ejecución controlada de comandos, la recopilación de información, la generación del resumen y la escritura del informe.

La versión final amplía el prototipo inicial con nuevas fuentes de información y medidas de robustez: ejecución mediante listas de argumentos, ausencia de intérprete de shell, límites temporales, tratamiento de errores y protección de `report.txt` mediante permisos 0600.

5.1.2 Estructura lógica del código

La implementación se organiza conceptualmente en cinco bloques: importaciones y configuración; ejecución controlada; funciones de recopilación; generación y protección del informe; y menú principal. Esta separación lógica facilita la comprensión y la ampliación futura.

Estructura lógica de LinuxRecon



Figura 15. Organización lógica de la implementación final de LinuxRecon. Fuente: elaboración propia.

5.1.3 Ejecución controlada de comandos

La función `run_command()` centraliza la ejecución de utilidades nativas. Los comandos se proporcionan como listas de argumentos y se ejecutan con `subprocess.run()`, sin utilizar `shell=True`. La función incorpora captura de salida, límites temporales y tratamiento específico de errores, evitando que un fallo puntual interrumpa necesariamente toda la recopilación.

```
result = subprocess.run(
    command,
    capture_output=True,
    text=True,
    timeout=timeout,
    check=False
)
```

5.1.4 Funciones de recopilación

Cada área del sistema se obtiene mediante una función específica. La Tabla 11 resume las principales funciones y la información asociada.

Función / componente	Información recopilada
<code>get_system_info()</code> / <code>get_distribution_info()</code>	Kernel, arquitectura, distribución y versión
<code>get_current_user()</code> / <code>get_user_id()</code> / <code>get_all_users()</code>	Usuario, UID, GID, grupos y cuentas
<code>get_user_aliases()</code>	Alias del usuario
<code>get_processes()</code>	Procesos activos
<code>get_open_ports()</code>	Sockets TCP/UDP
<code>get_network_info()</code> / <code>get_network_routes()</code>	Interfaces, direcciones y rutas
<code>get_mount_points()</code> / <code>get_disk_usage()</code>	Puntos de montaje y uso de disco
<code>get_suid_files()</code>	Archivos SUID
<code>get_recent_logs()</code> / <code>get_warning_logs()</code>	Eventos recientes y warnings
<code>generate_security_summary()</code>	Síntesis informativa

Tabla 11. Funciones principales implementadas en LinuxRecon. Fuente: elaboración propia.

5.1.5 Generación y protección del informe

Los resultados se integran en `report.txt` mediante secciones identificables: sistema, distribución, identidad, usuarios, alias, procesos, puertos, red, rutas, almacenamiento, SUID, eventos, warnings y resumen final. Tras la escritura, el archivo recibe permisos 0600, restringiendo su lectura y modificación al propietario.

Flujo de generación del informe



Figura 16. Flujo de recopilación, síntesis y protección de report.txt. Fuente: elaboración propia.

5.1.6 Interfaz y repositorio

La interfaz se mantiene deliberadamente sencilla. El usuario ejecuta `python3 my.py` y selecciona entre generar el informe completo o salir. Las opciones no válidas se rechazan sin provocar una excepción.

Repositorio oficial del proyecto: <https://github.com/Mirel/Linuxrecon>

5.2 Validación y pruebas

5.2.1 Metodología y entorno

La validación se realizó mediante pruebas funcionales y comparaciones directas con comandos nativos equivalentes. El entorno fue una máquina virtual Ubuntu 25.04 ARM64/AArch64 ejecutada mediante UTM sobre un Mac con Apple Silicon.

Elemento	Configuración
Equipo anfitrión	Mac con Apple Silicon
Virtualización	UTM
Sistema invitado	Ubuntu 25.04
Arquitectura	ARM64 / AArch64
Lenguaje	Python 3
Aplicación	LinuxRecon
Archivo principal	my.py
Salida	report.txt

Tabla 12. Entorno utilizado durante la validación de LinuxRecon. Fuente: elaboración propia.

5.2.2 Validación funcional

Se comprobó el inicio de la herramienta, la generación y regeneración del informe, la identidad del usuario, las rutas de red, los sockets en escucha, la enumeración de archivos SUID, el almacenamiento, los eventos, el resumen informativo, el tratamiento de entradas no válidas, la salida controlada, los permisos del informe y la corrección sintáctica del código.

```
mire@mire-QEMU-Virtual-Machine:~/linuxrecon$ ss -tln
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port
udp UNCONN 0 0 0.0.0.0:55511 0.0.0.0:*
udp UNCONN 0 0 192.168.66.5:3702 0.0.0.0:*
udp UNCONN 0 0 239.255.255.250:3702 0.0.0.0:*
udp UNCONN 0 0 127.0.0.54:53 0.0.0.0:*
udp UNCONN 0 0 127.0.0.53%lo:53 0.0.0.0:*
udp UNCONN 0 0 0.0.0.0:45908 0.0.0.0:*
udp UNCONN 0 0 0.0.0.0:5353 0.0.0.0:*
udp UNCONN 0 0 *:60781 *:60781
udp UNCONN 0 0 [fe80::38c2:8fff:fe70:8cad]%enp0s1:3702 [::]:*
udp UNCONN 0 0 [ff02::c]%enp0s1:3702 [::]:*
udp UNCONN 0 0 [::]:32772 [::]:*
udp UNCONN 0 0 [::]:5353 [::]:*
tcp LISTEN 0 4096 127.0.0.53%lo:53 0.0.0.0:*
tcp LISTEN 0 4096 127.0.0.1:631 0.0.0.0:*
tcp LISTEN 0 4096 127.0.0.54:53 0.0.0.0:*
tcp LISTEN 0 4096 [::]:631 [::]:*

mire@mire-QEMU-Virtual-Machine:~/linuxrecon$ grep -A12 "[PUERTOS ABIERTOS\]" report.txt
[PUERTOS ABIERTOS]
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port
udp UNCONN 0 0 0.0.0.0:55511 0.0.0.0:*
udp UNCONN 0 0 192.168.66.5:3702 0.0.0.0:*
udp UNCONN 0 0 239.255.255.250:3702 0.0.0.0:*
udp UNCONN 0 0 127.0.0.54:53 0.0.0.0:*
udp UNCONN 0 0 127.0.0.53%lo:53 0.0.0.0:*
udp UNCONN 0 0 0.0.0.0:45908 0.0.0.0:*
udp UNCONN 0 0 0.0.0.0:5353 0.0.0.0:*
udp UNCONN 0 0 *:60781 *:60781
udp UNCONN 0 0 [fe80::38c2:8fff:fe70:8cad]%enp0s1:3702 [::]:*
udp UNCONN 0 0 [ff02::c]%enp0s1:3702 [::]:*
udp UNCONN 0 0 [::]:32772 [::]:*
```

Figura 17. Comparación entre datos del sistema y resultados registrados por LinuxRecon. Fuente: elaboración propia.

```
mire@mire-QEMU-Virtual-Machine:~/linuxrecon$ grep -A12 "[PUERTOS ABIERTOS\]" report.txt
[PUERTOS ABIERTOS]
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port
udp UNCONN 0 0 0.0.0.0:55511 0.0.0.0:*
udp UNCONN 0 0 192.168.66.5:3702 0.0.0.0:*
udp UNCONN 0 0 239.255.255.250:3702 0.0.0.0:*
udp UNCONN 0 0 127.0.0.54:53 0.0.0.0:*
udp UNCONN 0 0 127.0.0.53%lo:53 0.0.0.0:*
udp UNCONN 0 0 0.0.0.0:45908 0.0.0.0:*
udp UNCONN 0 0 0.0.0.0:5353 0.0.0.0:*
udp UNCONN 0 0 *:60781 *:60781
udp UNCONN 0 0 [fe80::38c2:8fff:fe70:8cad]%enp0s1:3702 [::]:*
udp UNCONN 0 0 [ff02::c]%enp0s1:3702 [::]:*
udp UNCONN 0 0 [::]:32772 [::]:*
```

Figura 18. Puertos y sockets recopilados durante la validación. Fuente: elaboración propia.

```
mire@mire-QEMU-Virtual-Machine:~/linuxrecon$ grep -A15 "[INFORMACIÓN DE RED\]" report.txt
[INFORMACIÓN DE RED]
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 3a:c2:8f:70:8c:ad brd ff:ff:ff:ff:ff:ff
   altname enx3ac28f708cad
   inet 192.168.66.5/24 brd 192.168.66.255 scope global dynamic noprefixroute enp0s1
       valid_lft 3422sec preferred_lft 3422sec
   inet6 fdf4:9d8e:a49a:dd60:868a:b9a5:26b4:164c/64 scope global temporary dynamic
       valid_lft 602824sec preferred_lft 84379sec
   inet6 fdf4:9d8e:a49a:dd60:38c2:8fff:fe70:8cad/64 scope global dynamic mngtmpaddr proto kernel_ra
       valid_lft 2591960sec preferred_lft 604760sec
```

Figura 19. Información de red obtenida durante las pruebas. Fuente: elaboración propia.

5.2.3 Robustez y seguridad

La opción no válida produjo el mensaje «Opción no válida.» sin excepción, mientras que la opción 2 finalizó la herramienta de forma controlada. `python3 -m py_compile my.py` seguido de `echo $?` devolvió 0, confirmando la ausencia de errores sintácticos en la versión validada. Los permisos de `report.txt` se verificaron mediante `ls -l` y `stat`, obteniéndose modo 600 (rw-----). El recuento independiente de archivos SUID produjo 65 elementos, coincidiendo con el resumen final de LinuxRecon.

```
mire@mire-QEMU-Virtual-Machine:~/linuxrecon$ head -35 report.txt
=====
LinuxRecon Security Report
=====
Fecha: 2026-07-13 22:17:38.540314

[INFORMACIÓN DEL SISTEMA]
Linux mire-QEMU-Virtual-Machine 6.14.0-37-generic #37-Ubuntu SMP PREEMPT_DYNAMIC Fri Nov 14 23:05:04 UTC 2025 aarch64 aarch64 aarch64 GNU/Linux

[USUARIO ACTUAL]
mire

[ID DEL USUARIO]
uid=1000(mire) gid=1000(mire) grupos=1000(mire),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),100(users),115(lpadmin)

[USUARIOS DEL SISTEMA]
root
daemon
bin
sys
sync
games
man
lp
mail
news
uucp
proxy
www-data
backup
list
irc
```

Figura 20. Evidencia de creación y consulta del informe `report.txt`. Fuente: elaboración propia.

5.3 Matriz global de validación

ID	Funcionalidad validada	Resultado
P01	Inicio y menú	Superada
P02	Generación/regeneración de <code>report.txt</code>	Superada
P03	Sistema y distribución	Superada
P04	Identidad	Superada
P05	Rutas de red	Superada
P06	Puertos y sockets	Superada
P07	SUID	Superada
P08	Montajes y uso de disco	Superada
P09	Logs y warnings	Superada

ID	Funcionalidad validada	Resultado
P10	Resumen informativo	Superada
P11	Entrada no válida y salida	Superada
P12	Permisos 0600	Superada
P13	Compilación con py_compile	Superada

Tabla 13. Matriz global de validación de LinuxRecon. Fuente: elaboración propia.

Las trece pruebas fueron superadas satisfactoriamente. Dentro del entorno experimental no se detectaron errores funcionales que impidieran la ejecución de las capacidades implementadas.

5.4 Evaluación básica de eficiencia

5.4.1 Medición temporal de referencia

La medición temporal disponible corresponde a una versión base anterior a la ampliación final y se conserva únicamente como referencia histórica.

Métrica	Resultado
Tiempo real	16,719 s
Tiempo de usuario	0,315 s
Tiempo de sistema	1,789 s

Tabla 14. Medición temporal de referencia de la versión base. Fuente: elaboración propia.

5.4.2 Características del informe final

En la validación final ampliada, report.txt alcanzó aproximadamente 49 KB y 607 líneas.

Característica	Resultado
Tamaño aproximado	49 KB
Número de líneas	607
Formato	TXT
Permisos	0600

Tabla 15. Características del informe final ampliado. Fuente: elaboración propia.

```

mire@mire-QEMU-Virtual-Machine:~/linuxrecon$ python3 my.py
=====
LinuxRecon - TFM Tool
=====
1. Generar reporte completo
2. Salir
Selecciona una opción: 1
[+] Generando reporte...
[+] Reporte generado correctamente: report.txt
mire@mire-QEMU-Virtual-Machine:~/linuxrecon$ tail -30 report.txt
/snap/core22/1912/usr/bin/chfn
/snap/core22/1912/usr/bin/chsh
/snap/core22/1912/usr/bin/gpasswd
/snap/core22/1912/usr/bin/mount
/snap/core22/1912/usr/bin/newgrp
/snap/core22/1912/usr/bin/passwd
/snap/core22/1912/usr/bin/su
/snap/core22/1912/usr/bin/sudo
/snap/core22/1912/usr/bin/umount
/snap/core22/1912/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core22/1912/usr/lib/openssh/ssh-keysign
/snap/core22/1912/usr/libexec/polkit-agent-helper-1
/snap/core22/2340/usr/bin/chfn
/snap/core22/2340/usr/bin/chsh
/snap/core22/2340/usr/bin/gpasswd
/snap/core22/2340/usr/bin/mount
/snap/core22/2340/usr/bin/newgrp
/snap/core22/2340/usr/bin/passwd
/snap/core22/2340/usr/bin/su
/snap/core22/2340/usr/bin/sudo
/snap/core22/2340/usr/bin/umount
/snap/core22/2340/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core22/2340/usr/lib/openssh/ssh-keysign
/snap/core22/2340/usr/libexec/polkit-agent-helper-1
/snap/snapd/23772/usr/lib/snapd/snap-confine

=====
Fin del reporte
=====
mire@mire-QEMU-Virtual-Machine:~/linuxrecon$ █

```

Figura 21. Fragmento representativo del informe final generado por LinuxRecon. Fuente: elaboración propia.

5.4.3 Interpretación

La evidencia permite afirmar que LinuxRecon completa correctamente la recopilación y genera un informe manejable, pero no permite cuantificar formalmente una reducción de tiempo frente al procedimiento manual. Para ello sería necesario un experimento comparativo controlado y repetido.

5.5 Análisis global de los resultados

Las pruebas demuestran automatización, centralización, repetibilidad y consistencia de resultados seleccionados respecto a comandos nativos equivalentes. LinuxRecon recopila y sintetiza información, pero no clasifica automáticamente los elementos detectados como vulnerabilidades.

5.6 Relación entre hipótesis y resultados

Los resultados respaldan la hipótesis en los aspectos medidos: automatización, centralización, homogeneidad y consistencia. No se considera demostrada una reducción cuantitativa del tiempo ni una disminución estadística de errores humanos, ya que esas variables no fueron evaluadas con un diseño experimental específico.

5.7 Limitaciones de la demostración

La validación se realizó principalmente sobre Ubuntu 25.04 ARM64 en un entorno virtualizado. El comportamiento en otras distribuciones y arquitecturas no se ha comprobado experimentalmente. La evaluación es fundamentalmente funcional y no incluye análisis de CVE, explotación, CVSS, monitorización continua ni correlación avanzada.

5.8 Conclusiones de la demostración

La demostración confirma que LinuxRecon funciona como herramienta ligera de Information Gathering local. Las trece pruebas definidas fueron superadas y la solución satisface el alcance técnico establecido.

6. CONCLUSIONES Y LÍNEAS FUTURAS

6.1 Conclusiones generales

El TFM ha permitido diseñar, implementar y validar LinuxRecon como una herramienta orientada a automatizar la recopilación inicial de información en sistemas Linux. La versión final incorpora nuevas fuentes de información, un resumen informativo y medidas de seguridad sobre la ejecución y el informe.

El resultado es una aplicación ligera en Python que ejecuta de forma controlada operaciones de consulta, centraliza los resultados en report.txt y permite repetir un procedimiento homogéneo de reconocimiento.

6.2 Cumplimiento de los objetivos

El objetivo general se considera alcanzado. LinuxRecon automatiza información de sistema y distribución, identidad, cuentas, alias, procesos, sockets, interfaces y rutas, almacenamiento, SUID y eventos recientes. También genera un resumen y protege el informe con permisos 0600.

6.3 Contraste de la hipótesis

La hipótesis queda respaldada en los aspectos directamente verificados: centralización, homogeneización, reducción de operaciones manuales individuales y reproducibilidad. No se considera demostrada una reducción cuantitativa del tiempo frente a un procedimiento manual.

6.4 Aportaciones del proyecto

- Automatización de una secuencia de reconocimiento local mediante una única ejecución.
- Centralización de resultados en un informe estructurado.
- Ampliación funcional hacia distribución, alias, rutas, almacenamiento, logs y warnings.
- Resumen informativo sin clasificación automática de vulnerabilidades.
- Hardening mediante listas de argumentos, ausencia de shell, timeouts y gestión de errores.
- Protección del informe mediante permisos 0600.
- Repositorio versionado y documentación del proyecto.

6.5 Seguridad de la solución

La versión final evita `shell=True` y no acepta comandos arbitrarios del usuario. Las operaciones están predefinidas, se ejecutan con argumentos estructurados y cuentan con timeouts y gestión de excepciones. `report.txt` se protege con permisos 0600.

6.6 Eficiencia y reproducibilidad

LinuxRecon no requiere base de datos, agentes residentes, servidor web ni dependencias Python externas adicionales para sus funciones principales. La medición de 16,719 s se mantiene como referencia de la versión base, no como tiempo definitivo de la versión final.

6.7 Limitaciones

- Validación principal sobre Ubuntu ARM64.
- Dependencia de utilidades nativas de Linux.
- Información condicionada por los permisos del usuario.
- Código concentrado principalmente en `my.py`.
- Formato principal limitado a TXT.
- Ausencia de detección automática de vulnerabilidades.

6.8 Líneas futuras

Las mejoras prioritarias son la modularización física, una interfaz de plugins, exportación JSON/HTML, selección configurable de comprobaciones, pruebas automatizadas y validación multidistribución. También sería útil un experimento comparativo manual frente a automatizado.

6.9 Evolución propuesta

Versión	Objetivo	Estado
LinuxRecon 1.0	Prototipo de recopilación e informe TXT	Completado
LinuxRecon 1.x	Ampliación funcional y hardening	Completado en este TFM
LinuxRecon 2.0	Modularización, configuración y JSON/HTML	Futuro
LinuxRecon 3.0	Plugins e integraciones externas	Futuro

Tabla 16. Evolución propuesta de LinuxRecon. Fuente: elaboración propia.

6.10 Conclusión final

LinuxRecon demuestra que una fase inicial de reconocimiento local puede automatizarse mediante una herramienta ligera, reproducible y segura en su ejecución. Su valor principal reside en combinar simplicidad, automatización, trazabilidad y control, manteniendo separada la recopilación de la interpretación posterior de seguridad.

7. BIBLIOGRAFÍA

7.1 Ingeniería del software y metodología

Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

IEEE Computer Society. (2014). Guide to the Software Engineering Body of Knowledge (SWEBOK V3.0). IEEE.

7.2 Sistemas Linux y administración

The Linux Kernel Organization. (2026). Linux Kernel Documentation. <https://docs.kernel.org/>

Canonical Ltd. (2026). Ubuntu Documentation. <https://ubuntu.com/server/docs>

Shotts, W. E. (2019). The Linux Command Line (2nd ed.). No Starch Press.

7.3 Tecnologías utilizadas

Python Software Foundation. (2026). Python 3 Documentation. <https://docs.python.org/3/>

Git Project. (2026). Git Documentation. <https://git-scm.com/doc>

GitHub, Inc. (2026). GitHub Docs. <https://docs.github.com/>

Silva, M. C. (2026). LinuxRecon: herramienta para la recopilación automatizada de información en sistemas Linux. GitHub. <https://github.com/Mirel/Linuxrecon>

Microsoft. (2026). Visual Studio Code Documentation. <https://code.visualstudio.com/docs>

UTM Project. (2026). UTM Documentation. <https://docs.getutm.app/>

7.4 Herramientas de auditoría y ciberseguridad

Nmap Project. (2026). Nmap Reference Guide. <https://nmap.org/book/man.html>

CISOfy. (2026). Lynis Documentation. <https://cisofy.com/documentation/lynis/>

Greenbone AG. (2026). Greenbone Community Documentation. <https://greenbone.github.io/docs/latest/>

Wazuh, Inc. (2026). Wazuh Documentation. <https://documentation.wazuh.com/current/>

7.5 Informes y contexto de amenazas

Verizon. (2026). 2026 Data Breach Investigations Report. Verizon Business.

IBM. (2026). X-Force Threat Intelligence Index 2026. IBM Security.

8. ANEXOS

8.1 ANEXO A. Código fuente final de LinuxRecon (my.py)

El siguiente listado corresponde a la versión final validada durante el TFM.

```
import subprocess
from datetime import datetime
from pathlib import Path

REPORT_FILE = Path("report.txt")
COMMAND_TIMEOUT = 20

def run_command(command, timeout=COMMAND_TIMEOUT):
    if not isinstance(command, list) or not command:
        return "Error: formato de comando no válido.\n"
    try:
        result = subprocess.run(command, capture_output=True, text=True,
                                timeout=timeout, check=False)
        if result.stdout.strip():
            return result.stdout
        if result.stderr.strip():
            return f"Advertencia al ejecutar {' '.join(command)}:\n{result.stderr}"
        if result.returncode != 0:
            return f"El comando {' '.join(command)} finalizó con código {result.returncode}.\n"
        return "Sin resultados.\n"
    except FileNotFoundError:
        return f"Error: no se encontró el comando '{command[0]}'.\n"
    except subprocess.TimeoutExpired:
        return f"Error: el comando {' '.join(command)} superó el tiempo máximo de {timeout} segundos.\n"
    except Exception as e:
        return f"Error ejecutando comando: {e}\n"

def get_system_info(): return run_command(["uname", "-a"])
def get_current_user(): return run_command(["whoami"])
def get_user_id(): return run_command(["id"])
def get_processes(): return run_command(["ps", "aux"])
def get_open_ports(): return run_command(["ss", "-tuln"])
def get_network_info(): return run_command(["ip", "a"])
def get_network_routes(): return run_command(["ip", "route"])
def get_mount_points(): return run_command(["findmnt"])
def get_disk_usage(): return run_command(["df", "-h"])
def get_suid_files(): return run_command(["find", "/", "-perm", "-4000"],
                                           timeout=60)
def get_recent_logs(): return run_command(["journalctl", "-n", "30", "--no-pager"], timeout=30)
def get_warning_logs(): return run_command(["journalctl", "-p", "warning", "-n", "20", "--no-pager"], timeout=30)

def get_distribution_info():
    p = Path("/etc/os-release")
    try:
        return p.read_text(encoding="utf-8") if p.exists() else "No se encontró /etc/os-release.\n"
    except OSError as e:
        return f"Error leyendo /etc/os-release: {e}\n"

def get_all_users():
    try:
        with open("/etc/passwd", encoding="utf-8") as f:
            return "\n".join(line.split(":",1)[0] for line in f if ":" in line) + "\n"
    except OSError as e:
        return f"Error leyendo /etc/passwd: {e}\n"
```

```

def get_user_aliases():
    aliases=[]
    for fp in [Path.home()/.bashrc, Path.home()/.bash_aliases]:
        try:
            if fp.exists():
                for line in fp.read_text(encoding="utf-8",
errors="ignore").splitlines():
                    if line.strip().startswith("alias "):
                        aliases.append(f"{fp.name}: {line.strip()}")
                except OSError as e:
                    aliases.append(f"Error leyendo {fp}: {e}")
        return "\n".join(aliases)+"\n" if aliases else "No se encontraron alias
configurados.\n"

def generate_security_summary(system_info, distribution_info, open_ports,
suid_files, warning_logs):
    sockets=[l for l in open_ports.splitlines() if l.startswith(("tcp","udp"))]
    suids=[l for l in suid_files.splitlines() if l.startswith("/")]
    warns=[l for l in warning_logs.splitlines() if l.strip() and not
l.startswith(("Advertencia","Error"))]
    distribution="No identificada"
    for l in distribution_info.splitlines():
        if l.startswith("PRETTY_NAME="):
            distribution=l.split("=",1)[1].strip(' '); break
    arch="aarch64 / ARM64" if "aarch64" in system_info.lower() else ("x86_64" if
"x86_64" in system_info.lower() else "No identificada")
    return f"=====
RESUMEN DE SEGURIDAD
=====
Sistema:
  Distribución: {distribution}
  Arquitectura: {arch}

Resultados de recopilación:
  Sockets detectados: {len(sockets)}

  Archivos con permiso SUID: {len(suids)}
  Eventos warning recopilados: {len(warns)}

IMPORTANTE:
La presencia de un archivo SUID, un puerto abierto o un warning
no implica por sí misma la existencia de una vulnerabilidad.

Estado de la recopilación: COMPLETADA
=====
"""

def generate_report():
    system_info=get_system_info(); distribution_info=get_distribution_info()
    current_user=get_current_user(); user_id=get_user_id();
    all_users=get_all_users(); aliases=get_user_aliases()
    processes=get_processes(); open_ports=get_open_ports();
    network_info=get_network_info(); routes=get_network_routes()
    mounts=get_mount_points(); disk=get_disk_usage(); suids=get_suid_files();
    logs=get_recent_logs(); warnings=get_warning_logs()
    summary=generate_security_summary(system_info, distribution_info, open_ports,
suids, warnings)
    report=f"""LinuxRecon Security Report
Fecha: {datetime.now()}

[INFORMACIÓN GENERAL DEL SISTEMA]
{system_info}
[DISTRIBUCIÓN Y VERSIÓN]
{distribution_info}
[USUARIO ACTUAL]
{current_user}
[ID DEL USUARIO]
{user_id}
[USUARIOS DEL SISTEMA]
{all_users}
[ALIAS DEL USUARIO]
{aliases}

```

```

[PROCESOS EN EJECUCIÓN]
{processes}
[PUERTOS ABIERTOS]
{open_ports}
[INFORMACIÓN DE RED]
{network_info}
[RUTAS DE RED]
{routes}
[PUNTOS DE MONTAJE]
{mounts}

[USO DE SISTEMAS DE ARCHIVOS]
{disk}
[ARCHIVOS CON PERMISOS SUID]
{suids}
[LOGS RECIENTES]
{logs}
[WARNINGS DEL SISTEMA]
{warnings}
{summary}
Fin del reporte
"""
    try:
        REPORT_FILE.write_text(report, encoding="utf-8")
        REPORT_FILE.chmod(0o600)
        print(f"[+] Reporte generado correctamente: {REPORT_FILE}")
    except OSError as e:
        print(f"[-] Error al generar el reporte: {e}")

def main():
    print("=====")
    print("      LinuxRecon - TFM Tool")
    print("=====")
    print("1. Generar reporte completo")
    print("2. Salir")
    option=input("Selecciona una opción: ").strip()
    if option=="1": generate_report()
    elif option=="2": print("Saliendo...")
    else: print("Opción no válida.")

if __name__ == "__main__":
    main()

```

8.2 ANEXO B. Estructura y evidencia del informe final

En la validación final, report.txt alcanzó aproximadamente 49 KB y 607 líneas. Se presenta una síntesis representativa.

```

[INFORMACIÓN GENERAL DEL SISTEMA]
Linux ... aarch64 GNU/Linux

[DISTRIBUCIÓN Y VERSIÓN]
PRETTY_NAME="Ubuntu 25.04"
VERSION_ID="25.04"

[USUARIO ACTUAL]
mire

[RUTAS DE RED]
default via 192.168.66.1 dev enp0s1 ...
192.168.66.0/24 dev enp0s1 ...

[RESUMEN DE SEGURIDAD]
Distribución: Ubuntu 25.04
Arquitectura: aarch64 / ARM64
Sockets detectados: 16
Archivos con permiso SUID: 65
Eventos warning recopilados: 20

```

Estado de la recopilación: COMPLETADA

Permisos verificados del archivo: 600 (rw-----), propietario mire.

8.3 ANEXO C. Evidencias de validación

```
mire@mire-QEMU-Virtual-Machine:~/linuxrecon$ ss -tuln
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port
udp UNCONN 0 0 0.0.0.0:55511 0.0.0.0:*
udp UNCONN 0 0 192.168.66.5:3702 0.0.0.0:*
udp UNCONN 0 0 239.255.255.250:3702 0.0.0.0:*
udp UNCONN 0 0 127.0.0.54:53 0.0.0.0:*
udp UNCONN 0 0 127.0.0.53%lo:53 0.0.0.0:*
udp UNCONN 0 0 0.0.0.0:45908 0.0.0.0:*
udp UNCONN 0 0 0.0.0.0:5353 0.0.0.0:*
udp UNCONN 0 0 *:60781 **
udp UNCONN 0 0 [fe80::38c2:8fff:fe70:8cad]%enp0s1:3702 [::]:*
udp UNCONN 0 0 [ff02::c]%enp0s1:3702 [::]:*
udp UNCONN 0 0 [::]:32772 [::]:*
udp UNCONN 0 0 [::]:5353 [::]:*
tcp LISTEN 0 4096 127.0.0.53%lo:53 0.0.0.0:*
tcp LISTEN 0 4096 127.0.0.1:631 0.0.0.0:*
tcp LISTEN 0 4096 127.0.0.54:53 0.0.0.0:*
tcp LISTEN 0 4096 [::]:631 [::]:*
mire@mire-QEMU-Virtual-Machine:~/linuxrecon$ grep -A12 "[PUERTOS ABIERTOS\]" report.txt
[PUERTOS ABIERTOS]
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port
udp UNCONN 0 0 0.0.0.0:55511 0.0.0.0:*
udp UNCONN 0 0 192.168.66.5:3702 0.0.0.0:*
udp UNCONN 0 0 239.255.255.250:3702 0.0.0.0:*
udp UNCONN 0 0 127.0.0.54:53 0.0.0.0:*
udp UNCONN 0 0 127.0.0.53%lo:53 0.0.0.0:*
udp UNCONN 0 0 0.0.0.0:45908 0.0.0.0:*
udp UNCONN 0 0 0.0.0.0:5353 0.0.0.0:*
udp UNCONN 0 0 *:60781 **
udp UNCONN 0 0 [fe80::38c2:8fff:fe70:8cad]%enp0s1:3702 [::]:*
udp UNCONN 0 0 [ff02::c]%enp0s1:3702 [::]:*
udp UNCONN 0 0 [::]:32772 [::]:*
```

Anexo C.1. Comparación de resultados. Fuente: elaboración propia.

```
mire@mire-QEMU-Virtual-Machine:~/linuxrecon$ grep -A12 "[PUERTOS ABIERTOS\]" report.txt
[PUERTOS ABIERTOS]
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port
udp UNCONN 0 0 0.0.0.0:55511 0.0.0.0:*
udp UNCONN 0 0 192.168.66.5:3702 0.0.0.0:*
udp UNCONN 0 0 239.255.255.250:3702 0.0.0.0:*
udp UNCONN 0 0 127.0.0.54:53 0.0.0.0:*
udp UNCONN 0 0 127.0.0.53%lo:53 0.0.0.0:*
udp UNCONN 0 0 0.0.0.0:45908 0.0.0.0:*
udp UNCONN 0 0 0.0.0.0:5353 0.0.0.0:*
udp UNCONN 0 0 *:60781 **
udp UNCONN 0 0 [fe80::38c2:8fff:fe70:8cad]%enp0s1:3702 [::]:*
udp UNCONN 0 0 [ff02::c]%enp0s1:3702 [::]:*
udp UNCONN 0 0 [::]:32772 [::]:*
mire@mire-QEMU-Virtual-Machine:~/linuxrecon$
```

Anexo C.2. Puertos y sockets. Fuente: elaboración propia.

```
mire@mire-QEMU-Virtual-Machine:~/linuxrecon$ grep -A15 "[INFORMACIÓN DE RED\]" report.txt
[INFORMACIÓN DE RED]
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 3a:c2:8f:70:8c:ad brd ff:ff:ff:ff:ff:ff
    altname enx3ac28f708cad
    inet 192.168.66.5/24 brd 192.168.66.255 scope global dynamic noprefixroute enp0s1
        valid_lft 3422sec preferred_lft 3422sec
    inet6 fdf4:9d8e:a49a:dd60:868a:b9a5:26b4:164c/64 scope global temporary dynamic
        valid_lft 602824sec preferred_lft 84379sec
    inet6 fdf4:9d8e:a49a:dd60:38c2:8fff:fe70:8cad/64 scope global dynamic mngtmpaddr proto kernel_ra
        valid_lft 2591960sec preferred_lft 604760sec
mire@mire-QEMU-Virtual-Machine:~/linuxrecon$
```

Anexo C.3. Información de red. Fuente: elaboración propia.

```
mire@mire-QEMU-Virtual-Machine:~/linuxrecon$ head -35 report.txt

=====
LinuxRecon Security Report
=====
Fecha: 2026-07-13 22:17:38.540314

[INFORMACIÓN DEL SISTEMA]
Linux mire-QEMU-Virtual-Machine 6.14.0-37-generic #37-Ubuntu SMP PREEMPT_DYNAMIC Fri Nov 14 23:05:04 UTC 2025 aarch64 aarch64 aarch64 GNU/Linux

[USUARIO ACTUAL]
mire

[ID DEL USUARIO]
uid=1000(mire) gid=1000(mire) grupos=1000(mire),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),100(users),115(lpadmin)

[USUARIOS DEL SISTEMA]
root
daemon
bin
sys
sync
games
man
lp
mail
news
uucp
proxy
www-data
backup
list
irc
```

Anexo C.4. Generación y consulta de report.txt. Fuente: elaboración propia.